OAVS • TGT COMPUTER TEACHER RECRUITMENT EXAMINATION

Exception Handling

Comprehensive MCQ Question Bank — Java Programming

Syllabus Coverage (Strictly Limited)

- Types of Exceptions — Checked & Unchecked
- The **try** Keyword
- The **catch** Keyword
- The **finally** Keyword
- The **throws** Keyword
- Creating Your Own (Custom) Exception
- Exceptions and Inheritance

244	12	A–D	100%
TOTAL MCQs	SECTIONS	BALANCED OPTIONS	EXPLAINED

Prepared as an exam-oriented practice resource for OAVS TGT Computer Science aspirants • Topic: Exception Handling (Java) • Every question includes the correct answer and a concise explanation. Reviewed for duplicates and strictly limited to the prescribed syllabus.

Table of Contents

Sec	Section Title	Question Range	Count	Page
1	Introduction to Exception Handling	Q1–Q14	14 MCQs	3
2	Types of Exceptions	Q15–Q44	30 MCQs	6
3	The try Keyword	Q45–Q65	21 MCQs	12
4	The catch Keyword	Q66–Q88	23 MCQs	17
5	The finally Keyword	Q89–Q106	18 MCQs	22
6	The throws Keyword	Q107–Q126	20 MCQs	26
7	Creating Your Own Exception	Q127–Q149	23 MCQs	31
8	Exceptions and Inheritance	Q150–Q169	20 MCQs	36
9	Comparison Questions	Q170–Q185	16 MCQs	41
10	Code, Output and Compilation Questions	Q186–Q208	23 MCQs	45
11	Difficult and Tricky Questions	Q209–Q223	15 MCQs	52
12	Mixed OAVS TGT Practice Set	Q224–Q244	21 MCQs	57

How to Use This Question Bank

Each question card shows the question number, category label, and section topic, followed by four options (A–D), the correct answer in a highlighted strip, and a short explanation. Correct answers are randomly and evenly distributed across A, B, C and D — do not assume any positional pattern. Sections 1–8 build the syllabus topic-by-topic; Sections 9–10 focus on comparisons and code/output/compilation questions; Sections 11–12 provide difficult, tricky, and mixed practice questions in true OAVS TGT exam style. This edition has been checked for duplicate or overlapping questions, keeping only unique, exam-relevant items.

SEC 1 Introduction to Exception Handling

Q1–Q14 • 14 Questions

Q1 Definition Introduction to Exception Handling

What is an exception in Java?

A An event that disrupts the normal flow of a program's instructions during execution

B A comment ignored by the compiler

C A syntax error detected by the compiler

D A warning generated by the JVM before compilation

Ans: A

Explanation: An exception is a runtime event that disrupts the normal flow of program execution and can be handled using exception-handling constructs.

Q2 Concept Introduction to Exception Handling

Exception handling in Java is primarily used to:

A Separate error-handling code from regular business logic and prevent abnormal termination

B Reduce the size of the .class file

C Convert checked exceptions into syntax errors

D Increase the speed of the compiler

Ans: A

Explanation: Exception handling separates error-handling logic from normal code and prevents the program from terminating abnormally.

Q3 Concept Introduction to Exception Handling

Which of the following best describes 'abnormal termination' of a program?

A The program finishes after printing all output as intended

B The program stops suddenly because of an unhandled exception or error

C The program pauses for user input

D The program restarts automatically after completion

Ans: B

Explanation: Abnormal termination occurs when the JVM stops program execution because an exception was not handled.

Q4 Concept Introduction to Exception Handling

Java exception-handling mechanism mainly uses which set of keywords?

A try, catch, finally, throw, throws

B import, package, static, final

C extends, implements, super, this

D public, private, protected, default

Ans: A

Explanation: Java provides five keywords for exception handling: try, catch, finally, throw, and throws.

Q5 Concept Introduction to Exception Handling

In Java, all exception and error classes are ultimately subclasses of:

A Object class only

B RuntimeException class

C Exception class only

D Throwable class

Ans: D

Explanation: The Throwable class is the superclass of all errors and exceptions in Java's exception hierarchy.

Q6 **Concept** **Introduction to Exception Handling**

Which statement about Java's exception mechanism is correct?

A Exceptions cannot carry any information about the error

B Exceptions are objects that represent an error condition raised during program execution

C Exceptions can only be handled at compile time

D Exceptions are handled only by the operating system

Ans: B

Explanation: An exception in Java is represented as an object containing information about the error condition that occurred.

Q7 **Fill in the Blank** **Introduction to Exception Handling**

An exception object is _____ by the JVM or the program when an abnormal condition occurs.

A declared

B thrown

C compiled

D imported

Ans: B

Explanation: When an abnormal condition occurs, an exception object is created and thrown, either by the JVM or explicitly by the programmer.

Q8 **Conceptual** **Introduction to Exception Handling**

Which of the following is NOT a benefit of using exception handling?

A Guaranteed prevention of all logical errors in a program

B Ability to maintain normal program flow after handling an error

C Graceful recovery from runtime errors

D Separation of error-handling code from normal logic

Ans: A

Explanation: Exception handling manages runtime errors gracefully but cannot prevent logical errors in the program's design.

Q9 **Assertion-Reason** **Introduction to Exception Handling**

Assertion (A): Exception handling helps maintain the normal flow of a Java application.
Reason (R): Exception handling allows errors to be caught and processed without stopping the entire program.

A Both A and R are true but R is not the correct explanation of A

B Both A and R are true and R is the correct explanation of A

C A is true but R is false

D A is false but R is true

Ans: B

Explanation: Exception handling catches and processes runtime errors so that the program's normal flow is not abruptly broken, which is exactly why R correctly explains A.

Q10 **Concept** **Introduction to Exception Handling**

The default exception handler of the JVM does which of the following when an exception is not caught?

A Converts the exception into a warning

B Automatically retries the failed operation

C Silently ignores the exception

D Prints the exception name, message, and stack trace, then terminates the program

Ans: D

Explanation: The JVM's default handler prints the exception's stack trace to the console and terminates the currently executing thread.

Q11 **Concept** **Introduction to Exception Handling**

A 'stack trace' printed after an unhandled exception mainly shows:

A The memory address of the JVM

B The list of all classes imported in the program

C The list of variables declared in the program

D The sequence of method calls that led to the point where the exception occurred

Ans: D

Explanation: A stack trace lists the chain of method calls, in order, that led to the exception being thrown.

Q12 **Concept** **Introduction to Exception Handling**

Without exception handling, what generally happens when a runtime error occurs in a Java program?

A The line causing the error is automatically skipped

B The JVM prints a default error message and terminates the program abnormally

C The program continues execution silently

D The compiler halts and refuses to generate a .class file

Ans: B

Explanation: If an exception is not handled, the JVM's default exception handler prints a stack trace and terminates the program.

Q13 **Match the Following** **Introduction to Exception Handling**

Match the keyword in Column I with its primary role in Column II.
Column I: 1. try 2. catch 3. finally 4. throws
Column II: (a) Declares a checked exception in a method signature (b) Encloses risky code (c) Handles a thrown exception (d) Executes cleanup code regardless of outcome

A 1-d, 2-a, 3-b, 4-c

B 1-c, 2-b, 3-a, 4-d

C 1-a, 2-b, 3-c, 4-d

D 1-b, 2-c, 3-d, 4-a

Ans: D

Explanation: try encloses risky code (b), catch handles the exception (c), finally performs guaranteed cleanup (d), and throws declares a checked exception in the method signature (a).

Q14 **Difficult** **Introduction to Exception Handling**

Since the compiler does not force a programmer to handle unchecked exceptions, why can an unchecked exception still cause a program to crash just as abruptly as an unhandled checked exception?

A The JVM treats unchecked exceptions differently from checked ones when neither is caught

B Compile-time enforcement only affects whether handling is mandatory; if an unchecked exception is left uncaught at runtime, the JVM's default handler still terminates the program the same way

C Unchecked exceptions are automatically converted to warnings at runtime

D Unchecked exceptions never actually occur at runtime, so this situation cannot arise

Ans: B

Explanation: Being 'unchecked' only means the compiler does not require handling; if left genuinely uncaught at runtime, it reaches the same default JVM handler and causes the same abrupt termination as an unhandled checked exception.

SEC 2 Types of Exceptions

Q15–Q44 • 30 Questions

Q15 Classification Types of Exceptions

Java exceptions are broadly classified into:

A Local exceptions and global exceptions
B Public exceptions and private exceptions
C Static exceptions and dynamic exceptions
D Checked exceptions and unchecked exceptions

Ans: D

Explanation: Java exceptions are broadly divided into checked exceptions and unchecked (runtime) exceptions.

Q16 Definition Types of Exceptions

A checked exception is one that:

A Is checked by the compiler at compile time, forcing the programmer to handle or declare it
B Is automatically ignored by the compiler
C Can never be caught using a catch block
D Is checked by the JVM only at runtime and never at compile time

Ans: A

Explanation: Checked exceptions are verified by the compiler at compile time; the method must either handle them or declare them using throws.

Q17 Definition Types of Exceptions

An unchecked exception is also known as a:

A Logical exception
B Runtime exception
C Compile-time exception
D System exception

Ans: B

Explanation: Unchecked exceptions are also called runtime exceptions because they occur during program execution and are not checked at compile time.

Q18 Concept Types of Exceptions

Which class is the direct superclass of RuntimeException?

A Error
B Exception
C Object
D Throwable

Ans: B

Explanation: RuntimeException directly extends the Exception class.

Q19 Concept Types of Exceptions

Which of the following is an example of a checked exception?

A NullPointerException
B ArrayIndexOutOfBoundsException
C ArithmeticException
D IOException

Ans: D

Explanation: IOException is a checked exception because it does not extend RuntimeException; it must be handled or declared.

Q20 Concept **Types of Exceptions**

Which of the following is an example of an unchecked exception?

A InterruptedException
B ArithmeticException
C SQLException
D ClassNotFoundException

Ans: B

Explanation: ArithmeticException extends RuntimeException and hence is an unchecked exception.

Q21 Concept **Types of Exceptions**

NullPointerException belongs to which category of exception?

A Checked exception
B Syntax error
C Unchecked (runtime) exception
D Compile-time error

Ans: C

Explanation: NullPointerException is a subclass of RuntimeException, making it an unchecked exception.

Q22 Concept **Types of Exceptions**

Which of the following statements about checked exceptions is TRUE?

A They are always subclasses of RuntimeException
B They must be either caught or declared using throws, otherwise a compilation error occurs
C They can only occur inside a finally block
D They are ignored by the compiler completely

Ans: B

Explanation: If a checked exception is not handled or declared, the compiler generates an error, unlike unchecked exceptions.

Q23 Concept **Types of Exceptions**

Which of the following statements about unchecked exceptions is TRUE?

A They are not checked by the compiler at compile time
B They must always be declared using the throws keyword
C They cannot occur during program execution
D The compiler forces the programmer to handle them

Ans: A

Explanation: Unchecked exceptions are not verified by the compiler; handling them is optional, though still good practice.

Q24 Concept **Types of Exceptions**

ArrayIndexOutOfBoundsException occurs when:

A A network connection times out
B An array is accessed with an index outside its valid range
C A file is not found on the disk
D A class is not found during loading

Ans: B

Explanation: This unchecked exception is thrown when an array is accessed using an invalid (negative or too large) index.

Q25 Concept **Types of Exceptions**

ClassCastException is thrown when:

A An object is cast to a subclass or class it is not actually an instance of
B A method throws a checked exception
C An arithmetic operation divides by zero
D A file cannot be opened

Ans: A

Explanation: ClassCastException occurs at runtime when an invalid type cast is attempted between incompatible classes.

Q26 **Concept** **Types of Exceptions**

NumberFormatException typically occurs when:

A A string that is not a valid numeric format is parsed into a number
B A number is divided by zero
C A file is missing
D An array index is out of range

Ans: A

Explanation: NumberFormatException is thrown when methods like Integer.parseInt() receive a string that cannot be converted to a number.

Q27 **Classification** **Types of Exceptions**

Errors (subclasses of the Error class) in Java are generally considered:

A Checked exceptions that must be handled
B Warnings generated only during compilation
C The same as checked exceptions
D Conditions that are usually not expected to be caught or handled by applications

Ans: D

Explanation: Error and its subclasses (like OutOfMemoryError) represent serious problems that applications are usually not expected to handle.

Q28 **Concept** **Types of Exceptions**

Which of the following is true regarding compile-time checking of exceptions?

A The compiler enforces handling for all exceptions equally
B The compiler enforces handling only for checked exceptions
C The compiler never checks any exception
D The compiler checks only unchecked exceptions

Ans: B

Explanation: Only checked exceptions are enforced by the compiler; unchecked exceptions are not subject to this compile-time check.

Q29 **Identification** **Types of Exceptions**

Identify the unchecked exception among the following:

A IllegalArgumentException
B InterruptedException
C FileNotFoundException
D IOException

Ans: A

Explanation: IllegalArgumentException extends RuntimeException, so it is an unchecked exception; the others are checked exceptions.

Q30 **Identification** **Types of Exceptions**

Identify the checked exception among the following:

A NullPointerException
B ArrayIndexOutOfBoundsException
C ClassNotFoundException
D ArithmeticException

Ans: C

Explanation: ClassNotFoundException does not extend RuntimeException, so it is a checked exception that must be handled or declared.

Q31 **Misconception** **Types of Exceptions**

Which of the following is a common misconception about unchecked exceptions?

A They can occur at runtime
B They must always be declared with throws, otherwise the code will not compile
C They are always caused by programming logic errors
D They are subclasses of RuntimeException

Ans: B

Explanation: A common misconception is that unchecked exceptions must be declared with throws; in reality, declaring them is optional.

Q32 Concept Types of Exceptions

Which of the following best represents the position of RuntimeException in the exception hierarchy?

A Exception -> Throwable -> RuntimeException
B RuntimeException -> Throwable -> Exception
C Throwable -> Exception -> RuntimeException
D Throwable -> Error -> RuntimeException

Ans: C

Explanation: The correct hierarchy path is Throwable at the top, then Exception, and then RuntimeException as its subclass.

Q33 Concept Types of Exceptions

StackOverflowError is an example of:

A A checked exception
B An unchecked exception
C A compile-time exception
D An Error, not an Exception

Ans: D

Explanation: StackOverflowError is a subclass of Error, not Exception, and typically results from excessive recursion.

Q34 Example Types of Exceptions

Which situation is most likely to cause an ArithmeticException?

A Dividing an integer by zero
B Casting a String to an Integer object incorrectly
C Accessing an undeclared variable
D Opening a file that does not exist

Ans: A

Explanation: Integer division by zero throws an ArithmeticException with the message '/ by zero'.

Q35 Identification Types of Exceptions

Which of the following is a checked exception that is a subclass of another checked exception (IOException)?

A NullPointerException
B FileNotFoundException
C ArrayIndexOutOfBoundsException
D ArithmeticException

Ans: B

Explanation: FileNotFoundException extends IOException, so it is a checked exception; the other three are unchecked subclasses of RuntimeException.

Q36 Fill in the Blank Types of Exceptions

A _____ exception must be handled at compile time, or the code fails to compile.

A logical
B checked
C unchecked
D runtime

Ans: B

Explanation: Checked exceptions require compile-time handling; failing to handle or declare them causes a compilation error.

Q37 Concept Types of Exceptions

Which of the following correctly differentiates checked exceptions from Errors?

A Errors are always checked at compile time like checked exceptions
B Checked exceptions and Errors are the same category
C Checked exceptions extend Exception (excluding RuntimeException), while Errors extend the Error class
D Checked exceptions can never be caught, but Errors can

Ans: C

Explanation: Checked exceptions are subclasses of Exception (excluding RuntimeException), while Errors form a separate hierarchy under the Error class.

Q38 **Assertion-Reason** **Types of Exceptions**

Assertion (A): ArithmeticException is an unchecked exception.
Reason (R): ArithmeticException is a subclass of RuntimeException.

A A is true but R is false
B Both A and R are true but R is not the correct explanation of A
C Both A and R are true and R is the correct explanation of A
D A is false but R is true

Ans: C

Explanation: ArithmeticException extends RuntimeException, and all subclasses of RuntimeException are unchecked, which correctly explains the assertion.

Q39 **Match the Following** **Types of Exceptions**

Match the exception in Column I with its category in Column II.
Column I: 1. IOException 2. ArithmeticException 3. ClassNotFoundException 4. NullPointerException
Column II: (a) Unchecked (b) Checked (c) Checked (d) Unchecked

A 1-b, 2-a, 3-c, 4-d
B 1-b, 2-d, 3-a, 4-c
C 1-a, 2-b, 3-c, 4-d
D 1-c, 2-d, 3-a, 4-b

Ans: A

Explanation: IOException and ClassNotFoundException are checked exceptions, while ArithmeticException and NullPointerException are unchecked exceptions.

Q40 **Scenario** **Types of Exceptions**

A programmer writes code that may throw an IOException but does not handle or declare it. What happens?

A The program compiles but crashes at runtime
B The program fails to compile
C The program compiles and runs fine
D The IOException is automatically converted to a warning

Ans: B

Explanation: Since IOException is a checked exception, failing to handle or declare it results in a compile-time error.

Q41 **Scenario** **Types of Exceptions**

A programmer writes code that may throw a NullPointerException but does not handle it explicitly. What happens at compile time?

A The compiler automatically inserts a catch block
B The program compiles successfully because it is an unchecked exception
C The program fails to compile
D The program throws a syntax error

Ans: B

Explanation: Since NullPointerException is unchecked, the compiler does not force the programmer to handle or declare it.

Q42 **Concept** **Types of Exceptions**

Which of the following is the correct order from the most general to the most specific class?

A RuntimeException -> Exception -> Throwable
B Exception -> Throwable -> RuntimeException
C Throwable -> RuntimeException -> Exception
D Throwable -> Exception -> RuntimeException

Ans: D

Explanation: Throwable is the topmost class, followed by Exception, and then its subclass RuntimeException.

Q43 **Difficult** **Types of Exceptions**

Which of the following exceptions is NOT a subclass of RuntimeException?

A IllegalStateException
B InterruptedException
C ArithmeticException
D ArrayIndexOutOfBoundsException

Ans: B

Explanation: InterruptedException is a checked exception and does not extend RuntimeException, unlike the other three options.

Q44 **Misconception** **Types of Exceptions**

Which of the following is a common misconception about checked versus unchecked exceptions?

A Both categories ultimately extend Throwable
B Checked exceptions are verified by the compiler
C Unchecked exceptions include RuntimeException and its subclasses
D Checked exceptions are inherently more severe or dangerous than unchecked exceptions

Ans: D

Explanation: Being 'checked' or 'unchecked' only refers to compile-time enforcement, not actual severity; an unchecked NullPointerException can be just as serious as a checked IOException.

SEC 3 The try Keyword

Q45–Q65 • 21 Questions

Q45 Purpose The try Keyword

What is the purpose of the try block in Java?

A To enclose code that might throw an exception so it can be monitored
B To declare a method that may throw an exception
C To permanently stop program execution
D To import exception-handling classes

Ans: A

Explanation: The try block encloses the code that might generate an exception so that it can be monitored and handled.

Q46 Syntax The try Keyword

Which of the following is the correct basic syntax for a try block?

A try -> code -> catch(Exception e)
B try[code]catch(Exception e)
C try { // code } catch(Exception e) { // handler }
D try: // code :catch(Exception e)

Ans: C

Explanation: A try block uses curly braces and must be followed by at least one catch block or a finally block.

Q47 Rule The try Keyword

A try block in Java must be immediately followed by:

A Nothing; try can stand alone
B Only a catch block
C Only a finally block
D At least one catch block, a finally block, or both

Ans: D

Explanation: A try block cannot stand alone; it must be followed by at least one catch block, a finally block, or both.

Q48 Concept The try Keyword

Which of the following statements about the try block is correct?

A A try block can exist without any catch or finally block
B A try block must always contain a return statement
C A try block cannot contain another try block
D A try block can contain any valid Java statements, including another try block

Ans: D

Explanation: A try block can legally contain any valid statements, including nested try blocks.

Q49 Concept The try Keyword

What happens if no exception occurs inside a try block that has a catch block?

A The catch block still executes after the try block
B The finally block is also skipped
C The catch block is skipped and execution continues after the try-catch structure
D The program terminates immediately

Ans: C

Explanation: If no exception occurs, the catch block(s) are skipped, and normal execution continues after the try-catch structure.

Q50 **Concept** **The try Keyword**

Once an exception is thrown inside a try block, what happens to the remaining statements in that same try block?

A They execute only after the catch block finishes
B They are skipped, and control transfers to a matching catch block
C They cause a separate exception
D They continue to execute normally

Ans: B

Explanation: As soon as an exception is thrown, the remaining statements in the try block are skipped and control shifts to the matching catch block.

Q51 **Code/Output** **The try Keyword**

What is the output of the following code?

```
try {
  System.out.println("A");
  int x = 5/0;
  System.out.println("B");
} catch (ArithmeticException e) {
  System.out.println("C");
}
```

A A C
B C only
C A B C
D A B

Ans: A

Explanation: Statement 'A' prints, then the division by zero throws an ArithmeticException before 'B' can print, so control jumps directly to the catch block, printing 'C'.

Q52 **Code/Output** **The try Keyword**

What is the output of the following code?

```
try {
  System.out.println("Start");
} catch (Exception e) {
  System.out.println("Caught");
}
System.out.println("End");
```

A Caught End
B Start End
C Start
D Start Caught End

Ans: B

Explanation: Since no exception occurs in the try block, the catch block is skipped and execution proceeds to print 'Start' and then 'End'.

Q53 **Rule** **The try Keyword**

Which of the following is NOT allowed in Java regarding the try keyword?

A A try block followed only by a finally block
B A try block followed by multiple catch blocks
C A try block nested inside another try block
D A try block used alone without catch or finally

Ans: D

Explanation: A try block used alone, with no catch and no finally block, is a compilation error.

Q54 **Common Mistake** **The try Keyword**

A student writes:

```
try { int a = 10/0; }
```

with no catch or finally block. What is the result?

A The code fails to compile because try has no catch or finally block
B The code compiles and automatically catches the exception
C The code compiles and prints nothing
D The code compiles and throws an exception at runtime

Ans: A

Explanation: A try block without an accompanying catch or finally block causes a compile-time error in Java.

Q55 **Concept** **The try Keyword**

Which of the following can a try block directly enclose?

A Any valid Java statements, whether or not they can throw exceptions
B Only arithmetic operations
C Only variable declarations
D Only method calls that throw checked exceptions

Ans: A

Explanation: A try block can contain any valid Java code, not only statements capable of throwing exceptions.

Q56 **Concept** **The try Keyword**

If multiple statements inside a try block could throw different exceptions, and only one catch block matching one exception type is present, what happens if a different exception occurs?

A It is automatically caught by the existing catch block
B The compiler generates an error
C It goes uncaught by that catch block and propagates further if no other handler matches
D The program ignores the exception and continues

Ans: C

Explanation: An exception is only caught by a catch block whose type matches (or is a superclass of) the actual exception; otherwise, it propagates further.

Q57 **Scenario** **The try Keyword**

Why do developers place only the risky code (not the entire program logic) inside a try block?

A To keep the scope of exception monitoring focused, making the code more readable and efficient
B Because try blocks cannot contain loops
C Because Java does not allow long try blocks
D To speed up compilation

Ans: A

Explanation: Keeping try blocks focused on risky code improves readability and makes it clear which statements are being monitored for exceptions.

Q58 **Fill in the Blank** **The try Keyword**

The _____ block is used to enclose the code that is suspected to raise an exception.

A try
B catch
C finally
D throws

Ans: A

Explanation: The try block encloses code that is suspected of raising an exception, so it can be monitored.

Q59 Code/Output The try Keyword

What will happen when the following code executes?

```
try {
  String s = null;
  System.out.println(s.length());
} catch (NullPointerException e) {
  System.out.println("Null value found");
}
```

A It throws an uncaught exception
B It prints 'Null value found'
C It prints 0
D It fails to compile

Ans: B

Explanation: Calling length() on a null reference throws a NullPointerException, which is caught, printing 'Null value found'.

Q60 Concept The try Keyword

Which of the following best explains why try blocks are essential in exception handling?

A They automatically fix bugs in the code
B They mark the region of code where the JVM should watch for exceptions and route control to handlers
C They prevent all exceptions from occurring
D They replace the need for catch blocks entirely

Ans: B

Explanation: The try block defines the monitored region; if an exception occurs there, control is routed to an appropriate catch block.

Q61 Common Mistake The try Keyword

What is wrong with the following code?

```
try {
  int x = 10;
}
System.out.println(x);
```

A The try block has no catch or finally, causing a compile-time error
B x cannot be declared inside try
C Nothing is wrong; it compiles and runs fine
D System.out.println cannot follow a try block

Ans: A

Explanation: A lone try block without catch or finally is a compile-time error in Java, regardless of what is inside it.

Q62 Concept The try Keyword

Nested try blocks in Java mean:

A Multiple methods each having exactly one try block
B A try block placed inside another try block's body
C A try block placed after a catch block
D Two try blocks placed side by side

Ans: B

Explanation: Nested try blocks occur when one try block (with its own catch/finally) is placed inside the body of another try block.

Q63 Scenario The try Keyword

In a nested try-catch structure, if the inner try block does not catch a particular exception, what happens?

A The exception is automatically discarded
B The inner try block re-executes
C The program crashes immediately with no further checking
D The exception propagates to the outer try block to be checked against its catch blocks

Ans: D

Explanation: If the inner try-catch cannot handle the exception, it propagates outward to the enclosing try block for a possible match.

Q64 **Difficult** **The try Keyword**

Is the following code valid in Java?

```
try {
  // completely empty, no statements
} catch (ArithmeticException e) {
  System.out.println("Never runs");
}
```

A No, catch(ArithmeticException e) requires at least one arithmetic statement in try

B Yes, it compiles; an empty try block is legal even though the catch block can practically never execute

C No, an empty try block is always a compile-time error

D Yes, but only if a finally block is also added

Ans: B

Explanation: An empty try block is syntactically legal in Java; for unchecked exceptions like ArithmeticException, the compiler does not verify reachability, so the catch block compiles even though it can never actually trigger.

Q65 **Difficult** **The try Keyword**

A try block contains an infinite loop (one that never terminates) and no exception ever occurs inside it. What happens to any catch and finally blocks attached to it?

A They execute immediately in parallel with the loop

B Only the finally block executes, regardless of the loop

C They never execute, because control never leaves the try block as long as the loop runs forever

D The catch block executes once the loop reaches its 1000th iteration

Ans: C

Explanation: Since control must leave the try block (normally or via an exception) before catch or finally can run, an infinite loop with no exception means neither block ever executes.

SEC 4 The catch Keyword

Q66–Q88 • 23 Questions

Q66 **Purpose** **The catch Keyword**

What is the primary purpose of the catch block?

A To guarantee execution regardless of exceptions
B To declare exceptions thrown by a method
C To handle the exception thrown by the corresponding try block
D To terminate the JVM

Ans: C

Explanation: The catch block contains the code that handles the exception raised in its corresponding try block.

Q67 **Syntax** **The catch Keyword**

Which is the correct syntax for a catch block?

A catch Exception(e) { // handler code }
B catch: Exception e { // handler code }
C catch [Exception e] { // handler code }
D catch (Exception e) { // handler code }

Ans: D

Explanation: A catch block requires parentheses containing the exception type and a reference variable, followed by curly braces.

Q68 **Concept** **The catch Keyword**

The parameter declared in a catch block (e.g., 'e' in catch(Exception e)) is called the:

A Static handler
B Try variable
C Exception parameter
D Exception identifier

Ans: C

Explanation: The variable declared in the catch block's parentheses is referred to as the exception parameter.

Q69 **Concept** **The catch Keyword**

A catch block can only handle an exception if:

A The exception type in catch is unrelated to the thrown exception
B The catch block is placed before the try block
C The exception type in catch matches or is a superclass of the thrown exception
D The exception is declared as static

Ans: C

Explanation: A catch block handles an exception only if its declared type matches the thrown exception's type or is one of its superclasses.

Q70 **Rule** **The catch Keyword**

Can a single try block have multiple catch blocks in Java?

A Yes, multiple catch blocks can handle different exception types
B Only for checked exceptions
C No, Java allows only one catch block per try
D Only if there is no finally block

Ans: A

Explanation: Java permits multiple catch blocks after a single try block so that different exception types can be handled separately.

Q71 **Rule** **The catch Keyword**

When using multiple catch blocks, what is the correct ordering rule?

A Catch blocks must be arranged alphabetically

B Catch blocks for subclasses must appear before catch blocks for their superclasses

C Order does not matter at all

D Catch blocks for superclasses must appear before catch blocks for their subclasses

Ans: B

Explanation: More specific (subclass) exception catch blocks must be placed before more general (superclass) catch blocks, or a compile-time error occurs.

Q72 **Common Mistake** **The catch Keyword**

What error occurs if a catch(Exception e) block is placed before catch(ArithmeticException e) for the same try block?

A A compile-time error stating the ArithmeticException has already been caught / is unreachable

B Only a warning is generated, and the code compiles fine

C No error; both execute in sequence

D A runtime exception is thrown

Ans: A

Explanation: Since ArithmeticException is a subclass of Exception, placing the superclass catch first makes the subclass catch unreachable, causing a compile-time error.

Q73 **Code/Output** **The catch Keyword**

What is the output?

```
try {
  int[] arr = new int[3];
  arr[5] = 10;
} catch (ArithmeticException e) {
  System.out.println("Arithmetic");
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("Array");
}
```

A Array

B Compilation error

C Both print

D Arithmetic

Ans: A

Explanation: The array access is out of bounds, throwing ArrayIndexOutOfBoundsException, which matches the second catch block, printing 'Array'.

Q74 **Concept** **The catch Keyword**

If a catch block's exception type does not match the exception thrown, what happens?

A The finally block is skipped as well

B The catch block is skipped, and the exception propagates further

C A compile-time error occurs

D The catch block still executes

Ans: B

Explanation: A catch block only executes when its type matches the thrown exception; otherwise it is skipped, and the exception looks for another handler.

Q75 **Concept** **The catch Keyword**

Using catch(Exception e) as a single catch block can catch:

A Only Errors

B Both checked and unchecked exceptions, since Exception is their common superclass

C Only unchecked exceptions

D Only checked exceptions

Ans: B

Explanation: Since Exception is the superclass of both checked and unchecked exception types (excluding Error), catch(Exception e) can catch both.

Q76 **Concept** **The catch Keyword**

Can catch(Exception e) catch an OutOfMemoryError?

A Yes, because Error extends Exception

B Yes, but only in checked mode

C Yes, if finally is also present

D No, because Error and Exception are separate branches under Throwable

Ans: D

Explanation: Error and Exception are sibling subclasses of Throwable, so a catch block for Exception cannot catch an Error.

Q77 **Code/Output** **The catch Keyword**

What is the output?

```
try {
  String s = "abc";
  int n = Integer.parseInt(s);
} catch (NumberFormatException e) {
  System.out.println("Format issue");
} catch (Exception e) {
  System.out.println("General issue");
}
```

A Both print

B General issue

C Compilation error

D Format issue

Ans: D

Explanation: Parsing "abc" as an integer throws NumberFormatException, which matches the first (more specific) catch block, printing 'Format issue'.

Q78 **Concept** **The catch Keyword**

What is a 'multi-catch' block in Java (using the pipe symbol)?

A A catch block used only for RuntimeException

B A catch block that automatically rethrows exceptions

C A single catch block that can handle multiple unrelated exception types using '|'

D Multiple try blocks sharing one catch block

Ans: C

Explanation: A multi-catch block, written as catch (ExceptionType1 | ExceptionType2 e), lets one block handle multiple exception types.

Q79 **Common Mistake** **The catch Keyword**

What is incorrect about the following multi-catch declaration?

```
catch (ArithmeticException | ArithmeticException e)
```

A Nothing; it is valid syntax

B Multi-catch cannot repeat the same exception type; it causes a compile-time error

C Multi-catch cannot use the pipe symbol

D The variable name e is not allowed in multi-catch

Ans: B

Explanation: A multi-catch block cannot list the same exception type more than once, or the compiler reports an error.

Q80 **Concept** **The catch Keyword**

In a catch block, the exception parameter can be used to:

A Call methods like getMessage() and printStackTrace() to get error details

B Change the return type of the method

C Redeclare the try block

D Restart the JVM

Ans: A

Explanation: The exception object caught can be used to access details of the error using methods such as getMessage() or printStackTrace().

Q81 **Fill in the Blank** **The catch Keyword**

A catch block that specifies the type _____ can handle any exception because it is the superclass of all exception types.

A IOException
B Error
C Throwable or Exception
D RuntimeException

Ans: C

Explanation: Exception (or more broadly Throwable) is high enough in the hierarchy to catch a very wide range of exception types.

Q82 **Scenario** **The catch Keyword**

A try block might throw either IOException or ArithmeticException. What is the BEST practice for catch blocks?

A Use catch(Error e) for both
B Use a single catch(Exception e) block only
C Avoid using catch blocks and let the program crash
D Write separate, ordered catch blocks (or a multi-catch) for each specific exception type

Ans: D

Explanation: Best practice is to catch specific exception types separately (or via multi-catch) so that different errors can be handled appropriately.

Q83 **Code/Output** **The catch Keyword**

What is the output?

```
try {
  throw new ArithmeticException("Test");
} catch (ArithmeticException e) {
  System.out.println(e.getMessage());
}
```

A Compilation error
B Test
C ArithmeticException
D null

Ans: B

Explanation: The getMessage() method returns the message passed to the exception's constructor, which is 'Test' here.

Q84 **Concept** **The catch Keyword**

Which of the following is TRUE about catch block ordering when catching sibling exceptions (neither is a subclass of the other)?

A Only one sibling exception type can be caught per program
B Order does not matter since neither catches the other's exceptions
C The alphabetically first exception must be caught first
D Sibling exceptions cannot be caught in the same try-catch structure

Ans: B

Explanation: When exception classes are unrelated (siblings), the order of their catch blocks does not cause a compile error since neither shadows the other.

Q85 **Difficult** **The catch Keyword**

What happens if an exception is thrown inside a catch block itself?

A It propagates up, and if there is a finally block, that still executes before propagation completes
B The program hangs indefinitely
C It is automatically ignored
D The try block executes again

Ans: A

Explanation: An exception thrown inside a catch block behaves like any other exception: it propagates outward, but any associated finally block still executes first.

Q86 **Concept** **The catch Keyword**

What does 'rethrowing' an exception inside a catch block mean?

A Converting the exception into a compile-time error

B Catching the exception and then throwing it again (the same or a new exception) so an outer handler can process it

C Ignoring the exception completely

D Declaring the exception using throws instead of catching it

Ans: B

Explanation: Rethrowing means a catch block, after some local processing (like logging), uses throw to pass the exception (or a new one) up to an outer or calling handler.

Q87 **Code/Compilation** **The catch Keyword**

What happens in this code?

```
void process() throws IOException {
  try {
    readFile();
  } catch (IOException e) {
    System.out.println("Logging error");
    throw e;
  }
}
```

(readFile() declares throws IOException)

A It compiles but throw e is simply ignored at runtime

B It compiles: the exception is logged, then rethrown to the caller of process(), which is why process() itself declares throws IOException

C It fails to compile because throw e is not allowed inside a catch block

D It compiles only if e is renamed

Ans: B

Explanation: This is a valid rethrow pattern: the catch block logs the error and then rethrows it with 'throw e;' so the exception propagates to whoever calls process(), which is why process() must declare throws IOException.

Q88 **Difficult** **The catch Keyword**

A method is declared as 'void test() throws FileNotFoundException' (a checked exception). Inside the calling code, a programmer writes 'catch (SQLException e)' around the call to test(), where SQLException is an unrelated checked exception. What happens?

A It fails to compile, because the compiler can determine that test() cannot throw SQLException, making that catch block unreachable

B It compiles fine, since both are checked exceptions

C It compiles only if SQLException is renamed to match FileNotFoundException

D It compiles, but the catch block is simply ignored at runtime

Ans: A

Explanation: For checked exceptions, the compiler verifies that the corresponding try block can actually throw the declared type; since test() cannot throw the unrelated SQLException, this catch block causes a compile-time error.

SEC 5 The finally Keyword

Q89–Q106 • 18 Questions

Q89 **Purpose** **The finally Keyword**

What is the main purpose of the finally block?

A To declare checked exceptions

B To execute cleanup code regardless of whether an exception was thrown or caught

C To handle exceptions of a specific type

D To replace the catch block entirely

Ans: B

Explanation: The finally block is used for cleanup code (like closing resources) that must run whether or not an exception occurred.

Q90 **Syntax** **The finally Keyword**

Which is the correct syntax to attach a finally block to a try-catch structure?

A finally { } try { } catch(Exception e) { }

B try-finally { } catch(Exception e) { }

C try { } catch(Exception e) { } finally { }

D try { } finally { } catch(Exception e) { }

Ans: C

Explanation: The correct order is try block, followed by catch block(s), and finally the finally block at the end.

Q91 **Concept** **The finally Keyword**

Does a finally block execute if the try block completes successfully with no exception?

A Only if the try block contains a return statement

B Yes, finally always executes whether or not an exception occurs

C Only if a catch block is also present

D No, finally only runs when an exception occurs

Ans: B

Explanation: The finally block executes regardless of whether an exception was thrown, caught, or not thrown at all.

Q92 **Concept** **The finally Keyword**

If an exception is thrown in the try block and successfully caught, when does the finally block execute?

A It does not execute since the exception was already caught

B After the catch block finishes executing

C Only if the catch block also throws an exception

D Before the catch block starts executing

Ans: B

Explanation: The finally block executes after the try (and any matching catch) block completes, right before control leaves the try-catch-finally structure.

Q93 **Rule** **The finally Keyword**

Can a try block be followed by only a finally block, without any catch block?

A No, a catch block is mandatory

B Only for checked exceptions

C Yes, try-finally is valid even without a catch block

D Only inside static methods

Ans: C

Explanation: A try block can legally be followed by only a finally block, with no catch block at all.

Q94 **Concept** **The finally Keyword**

Which of the following can prevent a finally block from executing?

A Calling System.exit() inside the try or catch block
B A finally block always executes; nothing can prevent it
C A caught exception in the try block
D The try block completing normally

Ans: A

Explanation: Calling System.exit() terminates the JVM immediately, which is one of the few situations that prevents the finally block from executing.

Q95 **Code/Output** **The finally Keyword**

What is the output?

```
try {
  System.out.println("Try");
  int x = 10/0;
} catch (ArithmeticException e) {
  System.out.println("Catch");
} finally {
  System.out.println("Finally");
}
```

A Catch Finally
B Try Catch
C Try Catch Finally
D Try Finally

Ans: C

Explanation: The try block prints 'Try', the exception is caught printing 'Catch', and then the finally block always executes, printing 'Finally'.

Q96 **Code/Output** **The finally Keyword**

What is the output?

```
try {
  System.out.println("Try");
} finally {
  System.out.println("Finally");
}
System.out.println("After");
```

A Finally After
B Try After
C Try Finally After
D Try Finally

Ans: C

Explanation: Even with no catch block, finally executes after the try block completes; then the program continues to the next statement, printing 'After'.

Q97 **Difficult** **The finally Keyword**

What is the output?

```
public static int test() {
  try {
    return 1;
  } finally {
    System.out.println("Finally runs");
  }
}
```

When called and printed, what is displayed?

A 'Finally runs' is printed, then 1 is returned
B Compilation error due to return before finally
C 1 only, finally is skipped because of return
D Nothing is printed or returned

Ans: A

Explanation: Even when a try block contains a return statement, the finally block executes before the method actually returns the value.

Q98 Common Misconception The finally Keyword

Which of the following is a common misconception about the finally block?

A It never executes if the try block already handled the exception successfully

B It is mainly used for cleanup like closing files or connections

C It always executes after try and catch under normal circumstances

D It executes even when the try block has a return statement

Ans: A

Explanation: A common misconception is that finally is skipped once an exception is caught; in reality, it still executes after the catch block completes.

Q99 Concept The finally Keyword

Which scenario is one of the few exceptions where the finally block does NOT execute?

A The try block completes without any exception

B The catch block executes normally

C The JVM crashes or the program calls System.exit() during the try or catch block

D A checked exception is thrown in the try block

Ans: C

Explanation: A finally block will not execute if the JVM shuts down (e.g., via System.exit()) or crashes before reaching it.

Q100 Rule The finally Keyword

Which combination of blocks is INVALID in Java?

A try followed by catch

B try followed by catch and finally

C finally used alone without try

D try followed by finally

Ans: C

Explanation: A finally block cannot exist independently; it must always be associated with a try block.

Q101 Code/Output The finally Keyword

What is the output?

```
try {
  int[] a = {1,2,3};
  System.out.println(a[5]);
} catch (NullPointerException e) {
  System.out.println("NPE");
} finally {
  System.out.println("Cleanup");
}
```

A Cleanup only, then program crashes with an uncaught exception

B NPE only

C No output at all

D NPE Cleanup

Ans: A

Explanation: ArrayIndexOutOfBoundsException is thrown, but the catch block only matches NullPointerException, so it is not caught; finally still executes ('Cleanup') before the exception propagates and crashes the program.

Q102 Scenario The finally Keyword

Why is the finally block commonly used to close resources like files or database connections?

A Because it prevents exceptions from ever occurring

B Because it is faster than the catch block

C Because it guarantees execution regardless of success or failure, ensuring resources are released

D Because it executes only when no exception occurs

Ans: C

Explanation: Finally guarantees that cleanup code runs whether or not an exception occurred, which is ideal for releasing resources like files or connections.

Q103 **Concept** **The finally Keyword**

Can a finally block itself throw an exception?

A No, finally blocks cannot throw exceptions
B Only if the try block did not throw any exception
C Only checked exceptions can be thrown from finally
D Yes, a finally block can throw exceptions just like any other block

Ans: D

Explanation: A finally block can contain any valid code, including statements that may throw exceptions, just like a try or catch block.

Q104 **Fill in the Blank** **The finally Keyword**

The _____ block is guaranteed to execute after try and catch, except in cases like JVM shutdown.

A finally
B catch
C throws
D throw

Ans: A

Explanation: The finally block is designed to execute after the try-catch structure completes, almost always, except in rare cases like JVM termination.

Q105 **Difficult** **The finally Keyword**

What is the output?

```
try {
  System.out.println("A");
  throw new RuntimeException();
} finally {
  System.out.println("B");
}
```

(Assume no catch block and no surrounding handler)

A A only, program terminates silently
B A B, then the exception propagates and the program terminates abnormally
C A B, and the program continues normally afterward
D B only

Ans: B

Explanation: Even without a catch block, finally executes ('B') before the uncaught RuntimeException propagates and causes abnormal termination.

Q106 **Misconception** **The finally Keyword**

Which of the following is a common misconception about when the finally block runs?

A It runs after a matching catch block completes
B It runs even if the try block completes with no exception at all
C It runs only if a catch block is also present in the same structure
D It is associated with a try block, not a catch block alone

Ans: C

Explanation: A common misconception is that finally needs a catch block to run; in reality, a try-finally structure (with no catch at all) is valid, and finally still executes.

SEC 6 The throws Keyword

Q107–Q126 • 20 Questions

Q107 Purpose The throws Keyword

What is the purpose of the throws keyword in Java?

A To declare, in a method signature, the checked exceptions that method might throw

B To actually throw an exception object

C To catch an exception inside a method

D To create a new exception class

Ans: A

Explanation: The throws keyword is used in a method's signature to declare the checked exceptions it might throw, without handling them itself.

Q108 Syntax The throws Keyword

Which is the correct syntax for declaring a method that may throw an IOException?

A void readFile() throws IOException { }

B throws void readFile() IOException { }

C void readFile() throws (IOException) { }

D void readFile() throw IOException { }

Ans: A

Explanation: The throws keyword appears after the parameter list and before the method body, followed by the exception class name(s).

Q109 Concept The throws Keyword

The throws clause is mainly required for:

A Runtime exceptions only

B Checked exceptions

C Unchecked exceptions only

D Errors only

Ans: B

Explanation: The throws clause is primarily used to declare checked exceptions, since the compiler requires them to be handled or declared.

Q110 Concept The throws Keyword

Can a method declare multiple exceptions using throws?

A Yes, multiple exceptions can be declared, separated by commas

B No, only one exception can be declared per method

C Only if all exceptions are unchecked

D Only inside constructors

Ans: A

Explanation: A method can declare multiple checked exceptions in its throws clause, separated by commas.

Q111 Concept The throws Keyword

If method A() calls method B(), and B() declares 'throws IOException', what must method A() do?

A A() must either catch the IOException or declare it using its own throws clause

B A() must convert IOException into a RuntimeException

C A() must rename the exception

D Nothing; A() can ignore this completely

Ans: A

Explanation: Since IOException is checked, any calling method must either handle it with try-catch or propagate it further using throws.

Q112 Concept The throws Keyword

Declaring 'throws Exception' on a method:

A Throws an exception immediately when the method starts

B Is invalid in Java

C Prevents any exception from occurring in the method

D Signals that the method might throw Exception or any of its subclasses, without handling them itself

Ans: D

Explanation: Declaring throws Exception tells callers that the method might throw an Exception (or any of its subtypes) that they must handle or propagate.

Q113 Common Mistake The throws Keyword

Which of the following is a compilation error?

A A method that throws IOException internally but does not declare it and does not catch it

B A method with 'throws IOException' that never actually throws IOException

C A method that declares throws for an exception it partially handles

D A method with an empty throws clause listing zero exceptions

Ans: A

Explanation: If a method can throw a checked exception like IOException, it must either catch it or declare it using throws; failing to do either causes a compile-time error.

Q114 Concept The throws Keyword

Does declaring throws on a method actually handle the exception?

A No, throws prevents the exception from occurring

B Yes, throws fully handles the exception

C Yes, but only for RuntimeException

D No, throws only informs callers that the exception might occur; it does not handle it

Ans: D

Explanation: The throws clause is purely a declaration; the actual exception is not handled and must be dealt with by the calling method.

Q115 Concept The throws Keyword

Is it mandatory to use throws for unchecked exceptions like ArithmeticException?

A Only if the method is public

B Yes, it is always mandatory

C Only in static methods

D No, it is optional since unchecked exceptions are not checked at compile time

Ans: D

Explanation: Declaring throws for unchecked exceptions is optional because the compiler does not enforce handling for them.

Q116 Comparison The throws Keyword

What is the key difference between throws and try-catch?

A throws is used only for arrays; try-catch is used only for strings

B throws handles the exception; try-catch only declares it

C throws declares that a method might throw an exception; try-catch actually handles the exception

D They are functionally identical

Ans: C

Explanation: throws is a declaration mechanism placed in a method signature, while try-catch is used to actually handle an exception where it occurs.

Q117 Code/Compilation The throws Keyword

Will the following code compile?

```
void process() throws IOException {
  throw new IOException("error");
}
```

A No, IOException must be caught inside process()
B No, custom messages are not allowed in IOException
C No, void methods cannot declare throws
D Yes, because IOException is declared using throws, so the method does not need to catch it

Ans: D

Explanation: Since the checked exception is declared with throws, the method is not required to catch it internally; the caller becomes responsible.

Q118 Concept The throws Keyword

A constructor in Java:

A Can only throw unchecked exceptions
B Automatically catches all exceptions
C Can declare throws just like a regular method
D Cannot declare throws under any circumstances

Ans: C

Explanation: Constructors in Java can declare checked exceptions using throws, just as regular methods can.

Q119 Fill in the Blank The throws Keyword

A method that might throw a checked exception but does not want to handle it internally should use the _____ keyword in its signature.

A throw
B finally
C throws
D catch

Ans: C

Explanation: The throws keyword in a method signature declares checked exceptions the method may propagate to its caller.

Q120 Concept The throws Keyword

When overriding a method, can the overriding method declare broader checked exceptions than the overridden method?

A Only for unchecked exceptions, which have no restriction
B Only if the method is static
C Yes, any exception can be declared
D No, the overriding method cannot declare new or broader checked exceptions than the parent method

Ans: D

Explanation: Java restricts overriding methods from declaring broader or new checked exceptions than the method they override, to preserve substitutability.

Q121 Concept The throws Keyword

Overriding methods CAN freely declare:

A Broader checked exceptions than the parent method
B Fewer or no checked exceptions than the parent method, or any unchecked exceptions
C Only the exact same exceptions with no flexibility at all
D No exceptions of any type

Ans: B

Explanation: An overriding method may declare fewer checked exceptions (or none) than the parent method, and may freely declare unchecked exceptions.

Q122 **Difficult** **The throws Keyword**

If method X() declares 'throws IOException, SQLException', and its caller only catches IOException, what happens?

A SQLException is automatically ignored

B The code compiles fine since one exception is handled

C IOException handling also covers SQLException

D The code fails to compile because SQLException is still an unhandled checked exception

Ans: D

Explanation: Every checked exception declared by a called method must be handled or further declared; leaving SQLException unhandled causes a compile-time error.

Q123 **Concept** **The throws Keyword**

Which statement about the throws keyword and method signatures is TRUE?

A throws appears before the method's return type

B throws must appear inside the method body

C throws appears after the parameter list, before the method body

D throws replaces the return type entirely

Ans: C

Explanation: In Java syntax, the throws keyword and the list of declared exceptions appear after the parameter list and before the opening brace of the method body.

Q124 **Application** **The throws Keyword**

A payroll application calls a third-party library method that declares 'throws PayrollServiceException' (a checked exception). The developer wants the calling method to simply pass this responsibility up to whoever calls it, without handling it locally. What should the developer do?

A Wrap the call in an empty try-catch block that does nothing

B Declare 'throws PayrollServiceException' on the calling method as well, propagating the responsibility upward

C Rename the exception so it is no longer checked

D Ignore the compiler error, since Java allows skipping checked exceptions

Ans: B

Explanation: Declaring the same checked exception with throws on the calling method is the correct way to propagate responsibility to its own caller instead of handling it immediately.

Q125 **Common Mistake** **The throws Keyword**

A student assumes that because a method declares 'throws IOException', the exception is already handled and the calling code needs no further action. What is wrong with this assumption?

A throws only declares the possibility of the exception; the caller must still catch it or declare it further, or the code will not compile

B Nothing; throws fully handles the exception

C throws automatically converts IOException into an unchecked exception for the caller

D throws prevents IOException from ever being thrown

Ans: A

Explanation: A common mistake is treating throws as if it handles the exception; it only passes the responsibility to the caller, who must still catch or further declare the checked exception.

Q126 **Difficult** **The throws Keyword**

Method a() is declared as 'void a() throws Exception' and internally calls method b(), which is declared as 'void b() throws IOException'. Can a() call b() directly without wrapping it in a try-catch?

A No, a() must always catch IOException explicitly regardless of its own throws clause

B Yes, but only if b() is declared private

C Yes, because declaring the broader 'throws Exception' already covers any narrower checked exception, such as IOException, thrown by methods it calls

D No, because IOException and Exception are unrelated types

Ans: C

Explanation: Since IOException is a subclass of Exception, declaring the broader 'throws Exception' on a() is sufficient to cover the narrower checked IOException thrown internally by b(), so no local try-catch is required.

SEC 7 Creating Your Own Exception

Q127–Q149 • 23 Questions

Q127 Concept Creating Your Own Exception

Why would a programmer create a custom (user-defined) exception in Java?

A To replace all built-in exceptions

B Because Java does not provide any built-in exceptions

C To represent application-specific error conditions that built-in exceptions do not adequately describe

D To avoid using try-catch blocks entirely

Ans: C

Explanation: Custom exceptions let programmers create meaningful, application-specific error types that clearly represent domain-specific problems.

Q128 Concept Creating Your Own Exception

To create a custom checked exception, a class should:

A Implement the Throwable interface directly without extending anything

B Extend the Exception class (and not RuntimeException)

C Extend the RuntimeException class

D Extend the Error class

Ans: B

Explanation: Extending Exception (and not RuntimeException) creates a checked custom exception that callers must handle or declare.

Q129 Concept Creating Your Own Exception

To create a custom unchecked exception, a class should:

A Extend the Throwable interface

B Extend the RuntimeException class

C Extend the Object class

D Extend the Exception class

Ans: B

Explanation: Extending RuntimeException produces a custom exception that behaves as unchecked, similar to built-in runtime exceptions.

Q130 Syntax Creating Your Own Exception

Which of the following correctly declares a basic custom checked exception class?

A class MyException extends Exception { }

B class MyException implements Exception { }

C class MyException extends Throw { }

D class MyException throws Exception { }

Ans: A

Explanation: A custom checked exception class is created by extending the Exception class using the 'extends' keyword.

Q131 Syntax Creating Your Own Exception

Which constructor pattern is commonly used when creating a custom exception class?

A A constructor that must be declared static

B A constructor that overrides the main() method

C A constructor with no parameters allowed

D A constructor that calls super(message) to pass the error message to the parent class

Ans: D

Explanation: Custom exception classes typically define a constructor that accepts a message and passes it to the superclass constructor using super(message).

Q132 **Code** **Creating Your Own Exception**

What does the following code do?

```
class InvalidAgeException extends Exception {
  public InvalidAgeException(String message) {
    super(message);
  }
}
```

A Defines a custom checked exception with a message passed to the Exception superclass

B Defines a custom unchecked exception

C Overrides the built-in Exception class permanently

D Creates a syntax error because super() cannot be used here

Ans: A

Explanation: This defines a new checked exception class, InvalidAgeException, which passes its message to the Exception superclass constructor.

Q133 **Concept** **Creating Your Own Exception**

How is a custom exception actually raised (triggered) in code?

A By declaring it as a variable

B Using the throws keyword inside a method body

C Automatically by the JVM without any code

D Using the throw keyword with a new instance of the custom exception class

Ans: D

Explanation: A custom exception, like any exception, is raised using the throw keyword with a newly created instance: throw new InvalidAgeException("message");

Q134 **Code** **Creating Your Own Exception**

Given:

```
class InvalidAgeException extends Exception { InvalidAgeException(String m) { super(m); } }
```

Which line correctly throws this exception inside a method?

A throw new InvalidAgeException("Age must be positive");

B throws new InvalidAgeException("Age must be positive");

C InvalidAgeException("Age must be positive");

D catch InvalidAgeException("Age must be positive");

Ans: A

Explanation: The throw keyword, followed by a new instance of the exception class, is the correct way to raise a custom exception.

Q135 **Concept** **Creating Your Own Exception**

If a custom exception class extends Exception, and a method throws this custom exception without catching it, what must the method do?

A It must make the method private

B It must rename the exception class

C Nothing extra is required

D It must declare the custom exception using throws in its signature

Ans: D

Explanation: Since the custom exception is checked (extends Exception), the method must declare it using throws if it does not catch it internally.

Q136 **Concept** **Creating Your Own Exception**

If a custom exception class extends RuntimeException instead, is declaring it with throws mandatory?

A Yes, it is always mandatory

B No, it is optional since it behaves as an unchecked exception

C Only if the method is public

D Only if the class has multiple constructors

Ans: B

Explanation: Extending RuntimeException makes the custom exception unchecked, so declaring it with throws becomes optional, not mandatory.

Q137 **Common Mistake** **Creating Your Own Exception**

What is wrong with this code?

```
class MyException extends Exception { }
void test() {
  throw new MyException("error");
}
```

A Nothing is wrong; it compiles fine

B Custom exceptions cannot be thrown using 'throw'

C MyException cannot have a String argument

D It fails to compile because the checked MyException is neither caught nor declared with throws in test()

Ans: D

Explanation: Since MyException extends Exception (checked), the method test() must catch it or declare 'throws MyException'; otherwise, compilation fails.

Q138 **Concept** **Creating Your Own Exception**

Which of the following is an example of a well-designed custom exception name?

A Data

B InsufficientBalanceException

C MyClass1

D Test123

Ans: B

Explanation: Custom exception names conventionally describe the specific error condition and typically end with 'Exception', such as InsufficientBalanceException.

Q139 **Identification** **Creating Your Own Exception**

Which of the following is a valid reason to prefer a custom exception over reusing a generic built-in exception like Exception?

A Custom exceptions run faster than built-in exceptions

B Custom exceptions cannot be caught, making code simpler

C Custom exceptions provide clearer, more specific meaning about what went wrong in the application

D Custom exceptions eliminate the need for try-catch blocks

Ans: C

Explanation: Custom exceptions improve code clarity by giving specific, descriptive names to particular error conditions in an application.

Q140 **Fill in the Blank** **Creating Your Own Exception**

A custom exception class is typically created by extending either the _____ class (checked) or the _____ class (unchecked).

A Object, Exception

B Error, Throwable

C Exception, RuntimeException

D RuntimeException, Error

Ans: C

Explanation: Custom checked exceptions extend Exception, while custom unchecked exceptions extend RuntimeException.

Q141 **Scenario** **Creating Your Own Exception**

A banking application needs to signal that a withdrawal exceeds the account balance. What is the BEST approach?

A Create a custom exception, e.g., InsufficientFundsException, extending Exception or RuntimeException

B Use System.exit() to stop the program directly

C Use a generic Exception with no descriptive name

D Ignore the condition since Java has no way to represent it

Ans: A

Explanation: Creating a descriptively named custom exception like InsufficientFundsException is the standard, clear way to represent this specific business error.

Q142 **Code/Compilation** **Creating Your Own Exception**

Will the following code compile?

```
class MyException extends Exception {
  MyException(String m) { super(m); }
}
void test() throws MyException {
  throw new MyException("Custom error");
}
```

A No, because super(m) is invalid inside a custom exception

B No, because the constructor name must match 'Exception'

C No, because throw cannot be used with custom exceptions

D Yes, this compiles correctly since the checked exception is declared with throws

Ans: D

Explanation: The custom exception is properly declared using throws in the method signature, so this code compiles without error.

Q143 **Concept** **Creating Your Own Exception**

Custom exceptions should generally avoid:

A Extending Exception or RuntimeException

B Being named descriptively

C Introducing unnecessary complexity or overlapping heavily with existing built-in exceptions

D Providing a constructor that accepts a descriptive message

Ans: C

Explanation: Good practice avoids creating custom exceptions that needlessly duplicate the purpose of existing, well-understood built-in exceptions.

Q144 **Difficult** **Creating Your Own Exception**

What happens when you try to instantiate and throw a custom checked exception without ever defining a constructor in it?

A It compiles fine, since Java provides a default no-argument constructor automatically

B It fails to compile because a constructor is mandatory

C It compiles but always throws NullPointerException instead

D It cannot be thrown at all

Ans: A

Explanation: If no constructor is explicitly defined, Java automatically provides a default no-argument constructor, so the exception class still compiles and can be thrown with no message.

Q145 **Code** **Creating Your Own Exception**

A custom exception class is written with two constructors:

```
class ValidationException extends Exception {
  public ValidationException() { super(); }
  public ValidationException(String m) { super(m); }
}
```

What does this design allow?

A Only the no-argument constructor can ever be used

B This causes infinite recursion at runtime

C The exception can be thrown either with no message or with a specific descriptive message, just like many built-in exceptions

D It is invalid; a custom exception can only have one constructor

Ans: C

Explanation: Overloading constructors (no-argument and message-based) is a common, valid pattern that mirrors how built-in exceptions like Exception itself are designed, giving callers flexibility.

Q146 **Match the Following** **Creating Your Own Exception**

Match the custom exception scenario in Column I with the class it should most appropriately extend in Column II.
Column I: 1. An error that callers MUST explicitly handle or declare 2. An error that represents a programming mistake callers need not be forced to handle
Column II: (a) RuntimeException (b) Exception

A Both should extend Error
B 1-b, 2-a
C Both should extend Throwable directly
D 1-a, 2-b

Ans: B

Explanation: Extending Exception (checked) forces callers to handle or declare the exception, while extending RuntimeException (unchecked) leaves handling optional.

Q147 **Application** **Creating Your Own Exception**

An online exam portal needs to reject a submission if a candidate uploads a file larger than the allowed size, with a clear, application-specific error message. Which approach best fits Java's exception-handling model?

A Print an error to the console without using any exception mechanism
B Use only System.exit() with no exception class at all
C Define a custom exception such as FileSizeExceededException and throw it with a descriptive message when the condition is detected
D Let the program crash with a generic, unrelated built-in exception

Ans: C

Explanation: Creating a descriptively named custom exception and throwing it with a clear message is the standard, idiomatic way to represent and communicate this application-specific error condition.

Q148 **Common Mistake** **Creating Your Own Exception**

What is wrong with the following custom exception?

```
class MyException extends Exception {
  public MyException(String m) {
    // super(m) is missing here
  }
}
```

A The constructor parameter m becomes inaccessible
B It will not compile at all under any circumstances
C It automatically becomes an unchecked exception
D It compiles, but getMessage() will return null instead of the intended message, since the message was never passed to the Throwable superclass

Ans: D

Explanation: A common mistake is forgetting to call super(m); without it, the message is never stored in the Throwable superclass, so getMessage() returns null even though a message was provided.

Q149 **Difficult** **Creating Your Own Exception**

A custom checked exception class does not override toString(). If an uncaught instance of it (created with a message) reaches the JVM's default handler, what is typically printed first?

A Only the message, with no class name
B Only the word 'Exception' regardless of the actual class name
C Nothing is printed at all
D The fully qualified class name of the exception, followed by a colon and the message, inherited from Throwable's default toString()

Ans: D

Explanation: Since Throwable provides a default toString() that custom exceptions inherit unless overridden, the output is the exception's fully qualified class name followed by ': ' and its message.

SEC 8 Exceptions and Inheritance

Q150–Q169 • 20 Questions

Q150 Concept Exceptions and Inheritance

Java's exception classes are organized using which object-oriented concept?

A Polymorphism only, with no inheritance
B Encapsulation only
C Inheritance, forming a class hierarchy rooted at Throwable
D Interfaces only

Ans: C

Explanation: Java's exception classes form an inheritance hierarchy, with Throwable at the root and Exception/Error branching from it.

Q151 Concept Exceptions and Inheritance

In the exception hierarchy, which of these is the parent (superclass) of ArithmeticException?

A IOException
B RuntimeException
C Error
D Throwable directly

Ans: B

Explanation: ArithmeticException directly extends RuntimeException in Java's exception class hierarchy.

Q152 Concept Exceptions and Inheritance

A catch block written as catch(Exception e) can catch an object of type ArithmeticException because:

A ArithmeticException is, through inheritance, a subclass of Exception
B ArithmeticException and Exception are unrelated classes
C ArithmeticException overrides the Exception class
D Java allows catching any object regardless of type

Ans: A

Explanation: Due to inheritance, an ArithmeticException object 'is-a' Exception object, so a catch block for Exception can catch it as well.

Q153 Rule Exceptions and Inheritance

When arranging multiple catch blocks for a class hierarchy of exceptions, which order is correct?

A Order is irrelevant for hierarchy-related exceptions
B Alphabetical order regardless of hierarchy
C Subclass exceptions first, superclass exceptions later
D Superclass exceptions first, subclass exceptions later

Ans: C

Explanation: Subclass (more specific) exception types must be caught before their superclasses; otherwise, the subclass catch block becomes unreachable, causing a compile error.

Q154 Code/Compilation Exceptions and Inheritance

What is wrong with the following code?

```
try {
  // risky code
} catch (Exception e) {
  System.out.println("General");
} catch (ArithmeticException e) {
  System.out.println("Arithmetic");
}
```

A It throws a runtime exception when executed
B Nothing is wrong; it compiles and runs fine
C The order should be reversed only for checked exceptions
D Compile-time error: ArithmeticException has already been caught by Exception, making the second catch unreachable

Ans: D

Explanation: Because Exception is a superclass of ArithmeticException, placing it first makes the more specific catch block unreachable, resulting in a compile-time error.

Q155 **Concept** **Exceptions and Inheritance**

A custom exception class that extends an existing built-in exception class (like IOException) inherits:

A A completely separate, unrelated exception hierarchy

B Only the parent's method names but not their behavior

C The behavior and catchability of the parent class, so it can also be caught by a catch block for the parent type

D Nothing from the parent class

Ans: C

Explanation: Through inheritance, a custom exception extending a built-in exception class can be caught by any catch block matching its parent class type as well.

Q156 **Scenario** **Exceptions and Inheritance**

If class ChildException extends ParentException, and a try block throws a ChildException, can catch(ParentException e) catch it?

A Yes, because ChildException 'is-a' ParentException through inheritance

B Only if ParentException is unchecked

C Only if ChildException has no constructors

D No, only the exact same class can be caught

Ans: A

Explanation: Inheritance allows a subclass exception object to be caught by a catch block typed to any of its superclasses, including direct parents.

Q157 **Scenario** **Exceptions and Inheritance**

If class ChildException extends ParentException, and a try block throws a ParentException object (not ChildException), can catch(ChildException e) catch it?

A Yes, since they are related by inheritance

B Yes, but only for checked exceptions

C Yes, but only inside static methods

D No, a superclass object cannot be caught by a subclass-type catch block

Ans: D

Explanation: Catching works via the 'is-a' relationship from subclass to superclass, not the other way around; a ParentException object cannot be caught as a ChildException.

Q158 **Concept** **Exceptions and Inheritance**

Custom exceptions extending RuntimeException participate in inheritance such that:

A They cannot be caught at all

B They automatically become Errors

C They behave exactly like checked exceptions

D They inherit the unchecked nature of RuntimeException and can be caught by any RuntimeException or Exception catch block

Ans: D

Explanation: By extending RuntimeException, a custom exception inherits its unchecked status and can be caught using catch blocks for RuntimeException or its superclasses.

Q159 **Code/Output** **Exceptions and Inheritance**

What is the output?

```
class MyException extends RuntimeException {
  MyException(String m) { super(m); }
}
try {
  throw new MyException("Custom error");
} catch (RuntimeException e) {
  System.out.println(e.getMessage());
}
```

A RuntimeException
B Compilation error since MyException must be caught specifically
C Custom error
D null

Ans: C

Explanation: Since MyException extends RuntimeException, catch(RuntimeException e) can catch it, and getMessage() returns the message passed to the constructor.

Q160 **Misconception** **Exceptions and Inheritance**

Which of the following is a common misconception about exception inheritance?

A Any exception object can be caught by a subclass-type catch block regardless of its actual type
B Custom exceptions can extend built-in exception classes
C Catch block ordering matters when exceptions share a parent-child relationship
D A subclass exception object can be caught by a superclass-type catch block

Ans: A

Explanation: A common misconception is that a superclass exception object can be caught by a more specific subclass catch block; this is not true since catching follows the actual object's type upward, not downward.

Q161 **Concept** **Exceptions and Inheritance**

Which of the following is TRUE about catching multiple exceptions related by inheritance using catch(Throwable t)?

A It catches only Exception subclasses, not Errors
B It catches virtually everything, including both Exception and Error subclasses, since Throwable is the topmost class
C It only catches unchecked exceptions
D It is invalid syntax in Java

Ans: B

Explanation: Since Throwable is the root of the entire exception/error hierarchy, catch(Throwable t) can catch both Exception-based and Error-based objects.

Q162 **Scenario** **Exceptions and Inheritance**

A developer creates: class DatabaseException extends Exception, and further class ConnectionFailureException extends DatabaseException. Which catch block ordering is correct?

A catch(DatabaseException e) before catch(ConnectionFailureException e)
B Both catch blocks cannot exist in the same try-catch structure
C catch(ConnectionFailureException e) before catch(DatabaseException e)
D Order does not matter since both are custom exceptions

Ans: C

Explanation: Since ConnectionFailureException is more specific (a subclass), it must be caught first; otherwise, its catch block becomes unreachable.

Q163 **Concept** **Exceptions and Inheritance**

Why is understanding exception inheritance important when designing multiple catch blocks?

A It has no real impact on program behavior
B It only matters for unchecked exceptions
C It determines which catch block will actually handle a given exception and in what order they must be arranged
D It only affects performance, not correctness

Ans: C

Explanation: Exception inheritance directly determines both which catch block matches a thrown exception and the required ordering of multiple catch blocks.

Q164 **Difficult** **Exceptions and Inheritance**

Class A extends Exception. Class B extends A. Class C extends B. If an object of type C is thrown, which of these catch blocks (assuming only one is present) will successfully catch it?

A None of the listed catch blocks can catch it
B catch(B e), catch(A e), or catch(Exception e), any one of them individually
C Only catch(C e) can catch it; no ancestor class works
D catch(A e) only

Ans: B

Explanation: Because of inheritance, an object of type C 'is-a' B, 'is-a' A, and 'is-a' Exception, so any single catch block matching one of these ancestor types can catch it.

Q165 **Fill in the Blank** **Exceptions and Inheritance**

When a subclass exception object is thrown, it can be caught by a catch block for that exact class or any of its _____ classes.

A subclass
B superclass (ancestor)
C unrelated
D sibling

Ans: B

Explanation: A thrown exception object can be caught by a catch block matching its own class or any ancestor (superclass) class in the hierarchy.

Q166 **Concept** **Exceptions and Inheritance**

Custom exceptions and inheritance work together mainly to:

A Eliminate the need for the throw keyword
B Force every custom exception to be identical in structure
C Allow custom exceptions to be organized into meaningful hierarchies and caught flexibly at different levels of specificity
D Prevent custom exceptions from ever being caught

Ans: C

Explanation: Inheritance lets developers build hierarchies of related custom exceptions, enabling flexible catching at various levels of specificity.

Q167 **Match the Following** **Exceptions and Inheritance**

Match the exception class in Column I with its immediate superclass in Column II.
Column I: 1. ArithmeticException 2. RuntimeException 3. IOException 4. Exception
Column II: (a) Throwable (b) Exception (c) RuntimeException (d) Exception

A 1-c, 2-d, 3-d, 4-a
B 1-a, 2-b, 3-c, 4-d
C 1-d, 2-c, 3-a, 4-b
D 1-b, 2-a, 3-d, 4-c

Ans: A

Explanation: ArithmeticException extends RuntimeException; RuntimeException and IOException both extend Exception; and Exception itself extends Throwable.

Q168 **Misconception** **Exceptions and Inheritance**

Which of the following is a common misconception about catching two unrelated 'sibling' exception classes (where neither extends the other)?

A Both siblings may share a common ancestor higher up the hierarchy, such as Exception

B They can be caught using separate catch blocks in any order

C A multi-catch block can combine two true siblings using the pipe symbol

D A catch block for one sibling exception can also catch objects of the other sibling exception

Ans: D

Explanation: A common misconception is assuming sibling exception classes (neither a subclass of the other) can catch each other's objects; a catch block only matches its own type or that type's actual subclasses, not unrelated siblings.

Q169 **Difficult** **Exceptions and Inheritance**

Class P extends Exception, and class C extends P. A try block might throw either a P object or a C object depending on runtime conditions, and only 'catch (P e)' is present. If a C object is actually thrown, is it caught, and can the code tell from e's declared type alone whether the real object was P or C?

A It is caught (since C 'is-a' P), but e's declared type is P, so the code cannot tell from the declared type alone whether the actual object was P or C without an explicit runtime check

B It is not caught at all, since catch(P e) only matches exact P objects

C It causes a compile-time error because P and C are too closely related

D It is caught, and e's declared type automatically becomes C

Ans: A

Explanation: catch(P e) catches the C object through inheritance, but the compile-time (declared) type of e remains P; distinguishing the actual runtime type requires something like e.getClass() or an instanceof check.

SEC 9 Comparison Questions

Q170–Q185 • 16 Questions

Q170 Comparison Comparison Questions

What is the key difference between checked and unchecked exceptions?

A Checked exceptions never extend Exception

B Checked exceptions are verified by the compiler; unchecked exceptions are not

C There is no real difference; the terms are interchangeable

D Unchecked exceptions are verified by the compiler; checked exceptions are not

Ans: B

Explanation: The defining difference is that checked exceptions are enforced by the compiler (must be handled or declared), while unchecked exceptions are not.

Q171 Comparison Comparison Questions

Which statement correctly compares checked and unchecked exceptions regarding RuntimeException?

A Neither category relates to RuntimeException

B Both categories are subclasses of RuntimeException equally

C Unchecked exceptions are RuntimeException and its subclasses (plus Error)

D Checked exceptions are subclasses of RuntimeException

Ans: C

Explanation: Unchecked exceptions consist of RuntimeException and its subclasses (Errors are also unchecked, but form a separate branch).

Q172 Comparison Comparison Questions

Compare try and catch: which statement is correct?

A try monitors code for exceptions; catch handles the exception if one occurs

B catch monitors code for exceptions; try handles the exception

C try can exist only after catch, never before

D Both try and catch perform the exact same function

Ans: A

Explanation: The try block is used to monitor risky code, while the catch block is responsible for handling any exception that try block raises.

Q173 Comparison Comparison Questions

Which of these is a valid distinction between try and catch?

A try always requires catch and never finally

B catch can exist without try under all circumstances

C try can exist without catch (if finally is present); catch cannot exist without a preceding try

D Both are functionally identical and interchangeable

Ans: C

Explanation: A try block can exist with only a finally block (no catch), but a catch block can never exist without an associated try block.

Q174 Comparison Comparison Questions

Compare catch and finally: which is TRUE?

A Both catch and finally always execute under every circumstance

B catch and finally can never appear in the same try structure

C catch executes only when a matching exception occurs; finally executes regardless of exceptions

D finally executes only when an exception occurs; catch always executes

Ans: C

Explanation: catch only runs if a matching exception is thrown, while finally is designed to run in almost all cases, exception or not.

Q175 **Comparison** **Comparison Questions**

Which of the following best differentiates throws from try-catch?

A throws can only be used with unchecked exceptions

B throws declares a possible exception in the method signature; try-catch actually handles it where used

C They serve the exact same purpose and are interchangeable everywhere

D throws handles the exception at the point it occurs; try-catch declares it for the caller

Ans: B

Explanation: throws is a declarative mechanism used in method signatures, while try-catch is an active mechanism that actually processes the exception.

Q176 **Comparison** **Comparison Questions**

Compare a parent exception class and a child exception class in a catch hierarchy:

A Neither can catch objects of the other under any circumstance

B A catch block for the child class can catch objects of the parent class

C A catch block for the parent class can catch objects of the child class, but not vice versa

D Both catch identical sets of exception objects always

Ans: C

Explanation: Because of the 'is-a' relationship, a parent-type catch block can catch child-class exception objects, but the reverse is not true.

Q177 **Comparison** **Comparison Questions**

Which statement compares built-in exceptions with user-defined (custom) exceptions?

A Built-in exceptions cannot be caught using standard catch blocks

B Built-in exceptions are pre-defined by the Java API; custom exceptions are created by programmers for specific needs

C Custom exceptions can never extend built-in exception classes

D There is no functional difference between the two once compiled

Ans: B

Explanation: Built-in exceptions come with the Java class library (e.g., ArithmeticException), while custom exceptions are user-created classes for specific application needs.

Q178 **Comparison** **Comparison Questions**

Which of the following statements correctly contrasts checked exceptions with unchecked exceptions in terms of program design?

A There is no design-level difference between the two

B Checked exceptions typically represent recoverable conditions the caller should anticipate; unchecked exceptions often represent programming errors

C Checked exceptions are always faster to process than unchecked exceptions

D Unchecked exceptions must always be documented using throws

Ans: B

Explanation: By convention, checked exceptions represent conditions a well-written application should anticipate and recover from, while unchecked exceptions often indicate programming bugs.

Q179 **Comparison** **Comparison Questions**

Which comparison between try and finally is accurate?

A try must always be paired with finally, never with catch

B try and finally cannot appear in the same method

C finally executes before try begins

D finally is optional and used for guaranteed cleanup; try can be paired with catch, finally, or both

Ans: D

Explanation: finally is an optional block used for cleanup, while try can be paired with catch, finally, or both (but not neither).

Q180 **Comparison** **Comparison Questions**

How does exception handling using try-catch differ from simply declaring exceptions with throws, in terms of program flow?

A try-catch stops the propagation of the exception by handling it locally; throws allows the exception to propagate to the caller

B throws also stops the exception; there is no propagation difference

C Both stop propagation identically

D try-catch always propagates the exception further regardless of handling

Ans: A

Explanation: try-catch actually processes the exception, halting its propagation, whereas throws simply passes the responsibility of handling to the calling method.

Q181 **Comparison** **Comparison Questions**

Compare catching a superclass exception versus catching a subclass exception directly:

A Superclass catch blocks cannot compile

B Both approaches are always identical in scope

C Catching the superclass is broader and can catch multiple related exception types; catching the subclass is more specific

D Catching a subclass is broader than catching a superclass

Ans: C

Explanation: Catching a superclass type is broader, covering many related exception subclasses, while catching a specific subclass targets only that particular exception type.

Q182 **Comparison** **Comparison Questions**

Which of these correctly compares built-in exception ArithmeticException with a hypothetical custom exception InvalidTransactionException (extends Exception)?

A ArithmeticException is unchecked; InvalidTransactionException, extending Exception, is checked

B InvalidTransactionException cannot be thrown using the throw keyword

C Both are always unchecked exceptions

D Both are always checked exceptions

Ans: A

Explanation: ArithmeticException extends RuntimeException (unchecked), while InvalidTransactionException, extending Exception directly, is checked.

Q183 **Assertion-Reason** **Comparison Questions**

Assertion (A): try and catch always appear together in valid Java code.
Reason (R): A try block can be followed only by a finally block with no catch block at all.

A A is true but R is false

B Both A and R are true and R is the correct explanation of A

C Both A and R are true but R is not the correct explanation of A

D A is false but R is true

Ans: D

Explanation: Assertion A is false because try-finally (without catch) is valid Java; Reason R correctly states this fact, contradicting A.

Q184 **Comparison** **Comparison Questions**

Which is the correct comparison between throw and throws?

A throw and throws are the same keyword with different spelling conventions

B throw is used to actually raise an exception instance; throws is used in a method signature to declare possible exceptions

C throws raises an exception instance; throw declares exceptions in a method signature

D Both throw and throws can only be used with checked exceptions

Ans: B

Explanation: throw is a statement that actually raises a specific exception object, while throws is a declaration used in method signatures.

Q185 Difficult Comparison Questions

Which is the most accurate comparison between declaring a broad 'throws Exception' versus declaring each specific checked exception a method can actually throw?

A Declaring 'throws Exception' compiles for any checked exception but gives callers less precise information about what can actually go wrong, compared to declaring specific exception types

B A broad 'throws Exception' prevents the method from compiling unless no exceptions actually occur

C They are functionally identical in every respect, including the information given to callers

D Declaring specific exception types is always illegal if more than one can occur

Ans: A

Explanation: While both compile, declaring specific checked exceptions is considered better practice because it tells callers precisely what can go wrong, whereas a broad 'throws Exception' obscures that detail even though it is technically valid.

SEC 10 Code, Output and Compilation Questions

Q186–Q208 • 23 Questions

Q186 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
try {
  int[] arr = new int[2];
  arr[2] = 100;
} catch (Exception e) {
  System.out.println(e.getClass().getSimpleName());
}
```

A ArrayIndexOutOfBoundsException
B ArithmeticException
C ClassCastException
D NullPointerException

Ans: A

Explanation: Accessing index 2 of a size-2 array (valid indices 0-1) throws ArrayIndexOutOfBoundsException, whose simple class name is printed.

Q187 **Which Catch Executes** **Code, Output and Compilation Questions**

Which catch block executes?

```
try {
  Object o = "text";
  Integer i = (Integer) o;
} catch (NullPointerException e) {
  System.out.println("NPE");
} catch (ClassCastException e) {
  System.out.println("CCE");
}
```

A NPE
B Neither; compilation fails
C Both execute
D CCE

Ans: D

Explanation: Casting a String object to Integer is an invalid type cast, throwing ClassCastException, matching the second catch block.

Q188 **Does finally execute** **Code, Output and Compilation Questions**

Does the finally block execute in the following code?

```
try {
  return;
} finally {
  System.out.println("Cleanup");
}
```

A Only if an exception also occurs
B Yes, finally executes even when try contains a return statement
C No, return skips finally entirely
D It causes a compilation error

Ans: B

Explanation: The finally block executes even if the try block contains a return statement; the method returns only after finally completes.

Q189 **Is code valid** **Code, Output and Compilation Questions**

Is the following code valid in Java?

```
try {
  int x = 5;
}
```

A Yes, if x is declared final

B No, a try block must be followed by at least one catch block or a finally block

C Yes, a try block never requires catch or finally

D Yes, but only inside the main method

Ans: B

Explanation: A try block cannot stand alone in Java; it must be accompanied by a catch block, a finally block, or both.

Q190 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
try {
  System.out.println("1");
  throw new Exception("err");
} catch (Exception e) {
  System.out.println("2");
  return;
} finally {
  System.out.println("3");
}
```

(inside a method returning void)

A 1 2

B 1 3

C 1 2 3

D 1 3 2

Ans: C

Explanation: The try prints '1', the exception is caught printing '2', and even though catch has a return, finally still executes before the method returns, printing '3'.

Q191 **Which exception type** **Code, Output and Compilation Questions**

Which exception type is involved in the following code?

```
int[] data = new int[-3];
```

A ClassNotFoundException

B ArithmeticException

C IOException

D NegativeArraySizeException (an unchecked RuntimeException)

Ans: D

Explanation: Creating an array with a negative size throws NegativeArraySizeException at runtime; it is a subclass of RuntimeException, hence unchecked.

Q192 **Correctly Ordered** **Code, Output and Compilation Questions**

Which catch block ordering is correctly ordered (compiles without error)?

A catch(Exception e) then catch(ArithmeticException e)

B catch(Throwable t) then catch(Exception e)

C catch(RuntimeException e) then catch(ArithmeticException e)

D catch(ArithmeticException e) then catch(Exception e)

Ans: D

Explanation: Specific subclass exceptions (ArithmeticException) must be caught before their more general superclass (Exception) to avoid an unreachable code error.

Q193 **What does throws indicate** **Code, Output and Compilation Questions**

In the declaration 'void save() throws SQLException, IOException', what does the throws clause indicate?

A The method might throw either of these checked exceptions, and callers must handle or declare them

B The method will definitely throw both exceptions every time it runs

C The method automatically catches both exceptions internally

D The method cannot be called from another class

Ans: A

Explanation: The throws clause simply declares the possibility that these checked exceptions may occur; it does not guarantee they will, nor does it handle them.

Q194 **Which declaration is correct** **Code, Output and Compilation Questions**

Which of the following method declarations is syntactically correct for declaring multiple checked exceptions?

A void process() throws IOException, SQLException { }

B void process() throw IOException, SQLException { }

C void process() throws IOException & SQLException { }

D void process() throws { IOException, SQLException } { }

Ans: A

Explanation: Multiple checked exceptions in a throws clause are separated by commas, as in 'throws IOException, SQLException'.

Q195 **Custom exception creation** **Code, Output and Compilation Questions**

Which of the following correctly creates and throws a simple custom exception?

A class LowBalanceException throws Exception { }

B class LowBalanceException { } ... throws new LowBalanceException();

C exception LowBalanceException extends Exception { }

D class LowBalanceException extends Exception { LowBalanceException(String m){ super(m);} } ... throw new LowBalanceException("Low balance");

Ans: D

Explanation: This correctly defines a checked custom exception extending Exception with a message constructor, and raises it using throw with the new keyword.

Q196 **Superclass/subclass catch** **Code, Output and Compilation Questions**

What happens when the following executes?

```
class MyException extends RuntimeException {}
try {
  throw new MyException();
} catch (RuntimeException e) {
  System.out.println("Caught as RuntimeException");
}
```

A It prints 'Caught as RuntimeException' because MyException is a subclass of RuntimeException

B It throws an uncaught exception

C It fails to compile because MyException must be caught by its exact type

D Nothing is printed

Ans: A

Explanation: Since MyException extends RuntimeException, a catch block for RuntimeException successfully catches the MyException object due to inheritance.

Q197 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
try {
  try {
    int x = 5/0;
  } catch (NullPointerException e) {
    System.out.println("Inner catch");
  }
} catch (ArithmeticException e) {
  System.out.println("Outer catch");
}
```

A Compilation error
B Outer catch
C Inner catch
D Both print

Ans: B

Explanation: The inner catch does not match ArithmeticException, so it propagates to the outer try-catch, which does match, printing 'Outer catch'.

Q198 **Code/Compilation** **Code, Output and Compilation Questions**

Does the following code compile?

```
try {
  int x = 10;
} catch (ArithmeticException e) {
  System.out.println("Error");
}
```

A Yes, but it will throw an exception at runtime regardless
B No, unused catch blocks are illegal
C No, because no exception can actually occur in the try block
D Yes, it compiles; Java does not require that a checked pairing between try content and unchecked catch types be provable at compile time

Ans: D

Explanation: This compiles fine. For unchecked exceptions, Java does not verify at compile time whether the try block can actually produce that exception.

Q199 **Code/Compilation** **Code, Output and Compilation Questions**

Does the following code compile?

```
try {
  int x = 10;
} catch (IOException e) {
  System.out.println("Error");
}
```

A No, because IOException requires a finally block
B Yes, since IOException is unchecked
C No, because the compiler determines the try block cannot throw the checked IOException, making it unreachable
D Yes, always compiles regardless of exception type

Ans: C

Explanation: For checked exceptions, the compiler verifies that the try block could actually throw that exception; since it cannot throw IOException here, this is a compile-time error.

Q200 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
class A extends Exception {}
class B extends A {}
try {
  throw new B();
} catch (A e) {
  System.out.println("Caught as A");
} catch (B e) {
  System.out.println("Caught as B");
}
```

A Caught as B
B Both print
C Compilation error: unreachable catch block for B
D Caught as A

Ans: C

Explanation: Since catch(A e) appears before catch(B e), and B is a subclass of A, the second catch block becomes unreachable, causing a compile-time error.

Q201 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
class A extends Exception {}
class B extends A {}
try {
  throw new B();
} catch (B e) {
  System.out.println("Caught as B");
} catch (A e) {
  System.out.println("Caught as A");
}
```

A Caught as A
B Both print
C Compilation error
D Caught as B

Ans: D

Explanation: With the subclass B caught first, the code compiles correctly, and since a B object is thrown, it matches the first catch block, printing 'Caught as B'.

Q202 **Code/Compilation** **Code, Output and Compilation Questions**

Will the following code compile?

```
try {
  int x = 5/0;
} catch (ArithmeticException e) {
  System.out.println("Error1");
} catch (ArithmeticException e) {
  System.out.println("Error2");
}
```

A No, because ArithmeticException cannot be caught twice ever
B Yes, both catch blocks execute in order
C Yes, but only the first catch block ever executes
D No, duplicate catch blocks for the exact same exception type cause a compile-time error

Ans: D

Explanation: Having two catch blocks for the exact same exception type is illegal in Java and results in a compile-time error.

Q203 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
public class Test {
  static void check(int age) throws Exception {
    if (age < 18) throw new Exception("Underage");
    System.out.println("Valid age");
  }
  public static void main(String[] a) {
    try { check(15); } catch (Exception e) { System.out.println(e.getMessage()); }
  }
}
```

A Underage
B Valid age
C null
D Compilation error

Ans: A

Explanation: Since age (15) is less than 18, an Exception with the message 'Underage' is thrown and caught, printing that message.

Q204 **Code/Compilation** **Code, Output and Compilation Questions**

Which correction makes this code compile?

```
void connect() {
  throw new java.sql.SQLException("fail");
}
```

A Rename the method to main()
B Nothing needs to change; it already compiles
C Change SQLException to a lowercase name
D Add 'throws java.sql.SQLException' to the method signature, or wrap the throw in a try-catch

Ans: D

Explanation: Since SQLException is checked, the method must either declare it with throws or catch it internally to compile successfully.

Q205 **Fill in the Blank** **Code, Output and Compilation Questions**

In the code 'catch (ArithmeticException | NullPointerException e)', the pipe symbol represents a _____ catch block.

A nested
B final
C static
D multi-

Ans: D

Explanation: This is called a multi-catch block, allowing a single catch clause to handle more than one exception type.

Q206 **Code/Output** **Code, Output and Compilation Questions**

What is the output?

```
try {
  throw new ArithmeticException("Bad math");
} catch (ArithmeticException e) {
  System.out.println(e);
}
```

A Compilation error
B Bad math
C java.lang.ArithmeticException: Bad math
D null

Ans: C

Explanation: Printing an exception object directly calls its toString(), which by default prints the fully qualified class name followed by a colon and the message.

Q207 **Difficult** **Code, Output and Compilation Questions**

What is the output?

```
try {
  try {
    int x = 5/0;
  } finally {
    System.out.println("Inner finally");
  }
} catch (ArithmeticException e) {
  System.out.println("Outer catch");
}
```

(the inner try has no catch block, only finally)

A Compilation error since the inner try has no catch block

B Inner finally only, then the program crashes

C Outer catch only

D Inner finally, then Outer catch, since the inner finally runs before the exception propagates to the outer catch

Ans: D

Explanation: A try block can legally have only a finally block (no catch); the inner finally executes first as the exception propagates outward, and the outer catch then handles the ArithmeticException.

Q208 **Difficult** **Code, Output and Compilation Questions**

A parent class method has no throws clause at all: 'void process() { }'. Can an overriding method in a subclass be declared as 'void process() throws RuntimeException { }'?

A Yes, because the restriction on widening declared exceptions in an override applies only to checked exceptions, not to unchecked exceptions like RuntimeException

B No, because the parent method must also be updated to match

C No, overriding methods can never add any throws clause not present in the parent

D Yes, but only if RuntimeException is renamed

Ans: A

Explanation: Java's override restriction on declared exceptions applies only to checked exceptions; an overriding method remains free to declare any unchecked exception, such as RuntimeException, even if the parent declared none.

SEC 11 Difficult and Tricky Questions

Q209–Q223 • 15 Questions

Q209 Tricky **Difficult and Tricky Questions**

What is the output?

```
public static void main(String[] a) {
  try {
    System.out.println("A");
    System.exit(0);
  } finally {
    System.out.println("B");
  }
}
```

A A B, then repeats infinitely

B A B

C A only; finally does not execute because System.exit() terminates the JVM immediately

D B only

Ans: C

Explanation: System.exit() immediately shuts down the JVM, so the finally block never gets a chance to execute after 'A' is printed.

Q210 Tricky **Difficult and Tricky Questions**

What is the output?

```
static int test() {
  int x = 1;
  try {
    return x;
  } finally {
    x = 2;
  }
}
```

What value does test() return?

A 0

B 2

C 1

D Compilation error

Ans: C

Explanation: The return value 1 is evaluated and stored before finally executes; changing x afterward in finally does not affect the already-determined return value.

Q211 Tricky **Difficult and Tricky Questions**

What is the output?

```
static int test() {
  try {
    return 10;
  } finally {
    return 20;
  }
}
```

What value does test() return?

A Compilation error since two returns exist

B 20

C 10

D Both 10 and 20 are returned

Ans: B

Explanation: When a finally block contains its own return statement, it overrides the try block's return value; the method returns 20.

Q212 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
try {
  try {
    throw new ArithmeticException("inner");
  } finally {
    System.out.println("Inner finally");
  }
} catch (ArithmeticException e) {
  System.out.println("Outer catch: " + e.getMessage());
}
```

A Inner finally only

B Compilation error

C Outer catch: inner, Inner finally

D Inner finally, Outer catch: inner

Ans: D

Explanation: The inner finally executes first ('Inner finally') as the exception propagates outward, and then the outer catch handles it, printing the message.

Q213 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
try {
  throw new RuntimeException("first");
} finally {
  throw new RuntimeException("second");
}
```

(uncaught, at the top level)

A Only 'first' propagates; 'second' is ignored

B Neither exception propagates

C Both exceptions propagate together

D Only the exception thrown in finally ('second') propagates; the original exception ('first') is lost

Ans: D

Explanation: When a finally block throws its own exception, it replaces (and effectively suppresses) any exception that was propagating from the try block, so only 'second' propagates.

Q214 **Tricky** **Difficult and Tricky Questions**

Which of these correctly explains why catch(RuntimeException e) before catch(Exception e) is legal, but catch(Exception e) before catch(RuntimeException e) is not?

A Both orders are always legal in Java with no restriction

B Both orders are always illegal in Java

C RuntimeException is a subclass of Exception, so it must be caught first to remain reachable; the reverse order makes the RuntimeException catch unreachable

D Exception is a subclass of RuntimeException

Ans: C

Explanation: Since RuntimeException extends Exception, it is more specific and must appear first; placing Exception first would make the RuntimeException catch block unreachable.

Q215 **Tricky** **Difficult and Tricky Questions**

Given:

```
class LowStockException extends Exception {}
class CriticalStockException extends LowStockException {}
```

A method is declared as 'void check() throws LowStockException'. Can its body contain 'throw new CriticalStockException();' without any further throws declaration?

A No, because custom exceptions can never be thrown from a method that declares a different custom exception

B Yes, but only if CriticalStockException is changed to extend RuntimeException

C Yes, because CriticalStockException is a subclass of LowStockException, so declaring the ancestor type already covers it

D No, CriticalStockException must also be listed explicitly in the throws clause

Ans: C

Explanation: A throws clause listing a checked exception type also covers all of its checked subclasses, so a CriticalStockException (a LowStockException through inheritance) may be thrown without being listed separately.

Q216 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
try {
  System.out.println(1/0);
} catch (Exception e) {
  try {
    throw new NullPointerException("nested");
  } catch (ArithmeticException ex) {
    System.out.println("Inner");
  }
}
```

(assume the NullPointerException is not caught elsewhere)

A The program terminates abnormally with an uncaught NullPointerException, since the inner catch does not match its type

B A compile-time error occurs

C Inner is printed

D Nothing happens; both exceptions are silently ignored

Ans: A

Explanation: The outer ArithmeticException is caught, but inside the catch, a new NullPointerException is thrown; since the inner catch only matches ArithmeticException, the NullPointerException remains uncaught and propagates.

Q217 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
class MyException extends Exception {}
void test() throws MyException {
  // no actual throw statement here
}
```

Is this valid Java?

A No, MyException must extend RuntimeException to be declared this way

B No, a method declaring throws must always actually throw that exception

C No, this causes an infinite loop

D Yes, declaring throws for a checked exception is valid even if the method never actually throws it

Ans: D

Explanation: It is perfectly valid to declare throws for a checked exception in a method signature even if that exception is never actually thrown in the method body.

Q218 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
try {
  int[] arr = null;
  System.out.println(arr.length);
} catch (ArrayIndexOutOfBoundsException e) {
  System.out.println("Index issue");
} catch (NullPointerException e) {
  System.out.println("Null issue");
}
```

A Index issue
B Compilation error
C Null issue
D Both print

Ans: C

Explanation: Accessing .length on a null array reference throws NullPointerException, not ArrayIndexOutOfBoundsException, so the second catch block matches.

Q219 **Tricky** **Difficult and Tricky Questions**

A method overrides a parent method that declares 'throws IOException'. Which throws declaration is LEGAL for the overriding method?

A throws Throwable
B throws SQLException (an unrelated checked exception)
C throws Exception (a broader checked exception)
D throws FileNotFoundException (a subclass of IOException)

Ans: D

Explanation: An overriding method may narrow the declared checked exception to a subclass (like FileNotFoundException) but cannot widen it or declare unrelated checked exceptions.

Q220 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
try {
  for (int i = 0; i < 3; i++) {
    if (i == 1) throw new RuntimeException("stop");
    System.out.println(i);
  }
} catch (RuntimeException e) {
  System.out.println("Caught at i=1");
}
```

A Caught at i=1 only
B 0 1 2
C 0, Caught at i=1
D 0 1 Caught at i=1

Ans: C

Explanation: The loop prints 0, then on the next iteration (i=1) throws an exception before printing 1, so control jumps to the catch block, printing 'Caught at i=1'.

Q221 **Assertion-Reason** **Difficult and Tricky Questions**

Assertion (A): catch(Throwable t) can catch a StackOverflowError.
Reason (R): StackOverflowError, like all errors and exceptions, ultimately extends Throwable.

A A is false but R is true
B A is true but R is false
C Both A and R are true, but R does not explain A
D Both A and R are true, and R correctly explains A

Ans: D

Explanation: Since Throwable is the ultimate superclass of both Exception and Error hierarchies, catch(Throwable t) can indeed catch a StackOverflowError, and R correctly explains why.

Q222 **Tricky** **Difficult and Tricky Questions**

What is the output?

```
class MyException extends Exception {
  MyException(String m) { super(m); }
}
try {
  throw new MyException("custom failure");
} catch (MyException e) {
  e.printStackTrace();
  System.out.println("Handled");
}
```

What does printStackTrace() primarily do here?

A It stops the program immediately without printing 'Handled'

B It converts MyException into a RuntimeException

C It prints the exception type, message, and the method-call chain (stack trace) to the standard error stream, and then 'Handled' is printed next

D It silently does nothing in a catch block

Ans: C

Explanation: printStackTrace() outputs the exception's class name, message, and the sequence of method calls (stack trace) to System.err; execution then continues to the next statement, printing 'Handled'.

Q223 **Difficult** **Difficult and Tricky Questions**

Class X extends Exception. Class Y extends RuntimeException. A method declares 'throws X' only. Which statement about throwing a Y-type exception from that method is TRUE?

A It is illegal; the method can only throw exceptions it declares

B It causes a compile-time error regardless of Y's type

C It is legal only if Y also extends X

D It is legal; unchecked exceptions like Y do not need to be declared in throws, even if unrelated to X

Ans: D

Explanation: Since Y extends RuntimeException, it is unchecked and may be thrown from any method without being declared, regardless of what checked exceptions that method's throws clause lists.

SEC 12 Mixed OAVS TGT Practice Set

Q224–Q244 • 21 Questions

Q224 Mixed Mixed OAVS TGT Practice Set

Which keyword is used to explicitly throw an instance of an exception in Java?

A try
B throw
C throws
D catch

Ans: B

Explanation: The throw keyword is used with a new exception instance to explicitly raise (throw) that exception.

Q225 Mixed Mixed OAVS TGT Practice Set

Which of the following is the superclass of both Exception and Error?

A RuntimeException
B Serializable
C Throwable
D Object

Ans: C

Explanation: Throwable is the common superclass from which both Exception and Error are derived.

Q226 Mixed Mixed OAVS TGT Practice Set

If a try block has no matching catch block for a thrown exception but has a finally block, what happens?

A The try block is re-executed
B The finally block executes, and then the exception propagates further up the call stack
C The exception is automatically suppressed
D The finally block is skipped, and the program crashes immediately

Ans: B

Explanation: The finally block always executes before an unmatched exception continues propagating up the call stack.

Q227 Mixed Mixed OAVS TGT Practice Set

A method signature reads: 'int calculate(int a, int b) throws ArithmeticException'. Is declaring 'throws ArithmeticException' here necessary for compilation?

A Only if a and b are both zero
B Yes, absolutely mandatory
C Yes, but only for private methods
D No, it is optional since ArithmeticException is an unchecked exception

Ans: D

Explanation: Since ArithmeticException is unchecked, declaring it with throws is optional and only used here for documentation clarity.

Q228 Mixed Mixed OAVS TGT Practice Set

Which of the following best defines a 'custom exception'?

A An exception that can never be thrown
B A built-in exception renamed by the programmer
C A special kind of syntax error
D A user-defined class, typically extending Exception or RuntimeException, that represents an application-specific error

Ans: D

Explanation: A custom exception is a user-defined class, usually extending Exception or RuntimeException, created to model a specific application error condition.

Q229 **Mixed** **Mixed OAVS TGT Practice Set**

What is the output?

```
try {
  System.out.println("X");
} catch (ArithmeticException e) {
  System.out.println("Y");
} finally {
  System.out.println("Z");
}
```

A X Z
B X Y Z
C Z only
D X Y

Ans: A

Explanation: No exception occurs, so the catch block is skipped, but the finally block always executes, giving the output 'X Z'.

Q230 **Mixed** **Mixed OAVS TGT Practice Set**

Which of the following statements about multiple catch blocks is FALSE?

A They must be ordered from more specific to more general exception types
B Only the first matching catch block executes; the rest are skipped
C A try block can have any number of catch blocks
D All catch blocks always execute in sequence regardless of the exception type

Ans: D

Explanation: Only the single matching catch block executes for a given thrown exception; the remaining catch blocks are skipped, making this statement false.

Q231 **Mixed** **Mixed OAVS TGT Practice Set**

A custom exception class 'OutOfStockException' extends RuntimeException. Which of the following is TRUE?

A It automatically becomes a checked exception because it has a custom name
B It cannot be thrown using the throw keyword
C It must always be declared using throws wherever it might be thrown
D It behaves as an unchecked exception, so declaring it with throws is optional

Ans: D

Explanation: Since it extends RuntimeException, OutOfStockException is unchecked, so declaring it in a throws clause is optional, not required.

Q232 **Mixed** **Mixed OAVS TGT Practice Set**

What is the output?

```
try {
  Object obj = new Object();
  String s = (String) obj;
} catch (ClassCastException e) {
  System.out.println("Cast failed");
}
```

A Cast failed
B NullPointerException is thrown instead
C No output
D Compilation error

Ans: A

Explanation: Casting an Object instance directly to String is an invalid cast at runtime, throwing ClassCastException, which is caught here.

Q233 **Mixed** **Mixed OAVS TGT Practice Set**

In Java, which of these is the correct sequence of execution when an exception occurs inside a try block with matching catch and finally blocks present?

A Only finally executes; catch is skipped

B catch executes, then try resumes, then finally executes

C finally executes first, then catch, then remaining try statements

D Remaining try statements are skipped, matching catch executes, then finally executes

Ans: D

Explanation: Once an exception occurs, the rest of the try block is skipped, the matching catch block handles it, and finally executes afterward.

Q234 **Mixed** **Mixed OAVS TGT Practice Set**

Which of the following is generally considered good practice regarding custom exceptions and inheritance?

A Always make every custom exception extend Error

B Never allow custom exceptions to be caught

C Group related custom exceptions under a common custom superclass for flexible catching

D Avoid using constructors in custom exception classes

Ans: C

Explanation: Organizing related custom exceptions under a shared superclass allows callers to catch them individually or together at a higher level, improving flexibility.

Q235 **Mixed** **Mixed OAVS TGT Practice Set**

What is the output?

```
class InvalidInputException extends Exception {
  InvalidInputException(String m) { super(m); }
}
public class Test {
  static void validate(int n) throws InvalidInputException {
    if (n < 0) throw new InvalidInputException("Negative not allowed");
  }
  public static void main(String[] a) {
    try { validate(-5); } catch (InvalidInputException e) { System.out.println(e.getMessage()); }
  }
}
```

A null

B Compilation error

C -5

D Negative not allowed

Ans: D

Explanation: Since -5 is negative, the custom InvalidInputException is thrown with the given message, which is then caught and printed.

Q236 **Mixed** **Mixed OAVS TGT Practice Set**

Which of the following is TRUE regarding the relationship between Exception classes and inheritance in a multi-catch scenario?

A A multi-catch block using 'A | B' requires that A and B not be related by inheritance (neither a subclass of the other)

B Multi-catch blocks are unrelated to inheritance rules

C A multi-catch block can freely combine a class and its own subclass without restriction

D Multi-catch blocks can only include unchecked exceptions

Ans: A

Explanation: In a multi-catch block, the listed exception types must not be in a subclass-superclass relationship with each other, since that would make the combination redundant.

Q237 **Mixed** **Mixed OAVS TGT Practice Set**

What is the correct order of these blocks in a full try-catch-finally structure?

A try, catch, finally
B try, finally, catch
C finally, try, catch
D catch, try, finally

Ans: A

Explanation: The correct and only valid order in Java is: try block first, followed by catch block(s), and finally the finally block.

Q238 **Mixed** **Mixed OAVS TGT Practice Set**

Which statement is TRUE about exceptions thrown by the JVM itself (such as NullPointerException) versus exceptions thrown explicitly by a programmer using 'throw'?

A Only programmer-thrown exceptions can be caught by catch blocks
B Both JVM-thrown and programmer-thrown exceptions can be caught using appropriately typed catch blocks
C JVM-thrown exceptions can never be caught
D They belong to entirely separate, incompatible hierarchies

Ans: B

Explanation: Regardless of whether the JVM or the programmer throws an exception, it is still an object of an exception class and can be caught using a matching catch block.

Q239 **Mixed** **Mixed OAVS TGT Practice Set**

What is the output?

```
try {
  throw new Exception("Checked issue");
} catch (RuntimeException e) {
  System.out.println("Runtime");
}
```

A Runtime
B Checked issue
C Compilation error: the checked Exception is not caught by this catch block and is not declared or handled
D No output

Ans: C

Explanation: Since Exception is not a subclass of RuntimeException, this catch block cannot handle it; because it is a checked exception left unhandled, this results in a compile-time error.

Q240 **Mixed** **Mixed OAVS TGT Practice Set**

Which of the following best summarizes when a finally block will NOT run?

A When the JVM exits abruptly, such as via System.exit() or a fatal crash
B When the try block completes normally
C When a catch block also exists
D When an exception is caught successfully

Ans: A

Explanation: The primary situations where finally is skipped involve abrupt JVM termination, such as calling System.exit() or an unexpected crash of the JVM itself.

Q241 **Mixed** **Mixed OAVS TGT Practice Set**

A custom exception 'AgeLimitExceededException' should ideally extend which class if the application wants callers to be forced to handle it explicitly?

A RuntimeException
B Error
C Object
D Exception (making it checked)

Ans: D

Explanation: Extending Exception directly (not RuntimeException) makes the custom exception checked, forcing callers to handle or declare it.

Q242 **Mixed** **Mixed OAVS TGT Practice Set**

Which of the following is an example of a compile-time (checked) exception scenario that OAVS TGT candidates should recognize?

A Using an invalid array index without any catch block

B Calling a method that declares 'throws IOException' without handling or declaring it further

C Dividing an integer by zero without any catch block

D Accessing a null reference's method without any catch block

Ans: B

Explanation: Only the IOException scenario involves a checked exception, which forces a compile-time requirement to handle or declare it; the others are unchecked and only affect runtime behavior.

Q243 **Mixed** **Mixed OAVS TGT Practice Set**

A catch block catches an exception, logs it, and then executes 'throw new RuntimeException("Wrapped: " + e.getMessage());'. What is this pattern commonly called?

A Exception compilation

B Exception declaration

C Exception wrapping/rethrowing as a different exception type

D Exception suppression

Ans: C

Explanation: This is exception wrapping: the original exception is caught, logged, and a new exception (often unchecked) is thrown in its place to simplify the method's throws signature for callers.

Q244 **Difficult** **Mixed OAVS TGT Practice Set**

A try block may throw either a checked DatabaseException or an unchecked RuntimeException. Only one catch block is present: 'catch (RuntimeException e)'. If a DatabaseException (extends Exception directly, not RuntimeException) occurs, what happens?

A It causes catch(RuntimeException e) to catch it only during the first occurrence

B It is caught by catch(RuntimeException e) since both are exceptions

C It is NOT caught by catch(RuntimeException e), because DatabaseException is not a subclass of RuntimeException; this also means the checked DatabaseException is left unhandled, risking a compile-time error if not declared elsewhere

D It is automatically converted into a RuntimeException at runtime

Ans: C

Explanation: catch(RuntimeException e) only catches RuntimeException and its subclasses; since DatabaseException extends Exception directly, it is unrelated and must be separately caught or declared, or the checked-exception rule is violated.